

Institut Saint-Laurent

Enseignement supérieur économique de type court et de promotion sociale
Rue Saint-Laurent, 33 — 4000 Liège

TRAVAIL DE FIN D'ÉTUDES

CINAF v2

Plateforme de streaming dédiée au cinéma africain indépendant

« Le cinéma africain à portée de clic »

TAKOUMBO FABRICE AIME
Année académique 2025-2026

Travail de fin d'études présenté en vue de
l'obtention du diplôme de

Bachelier en informatique de gestion

Au moment de refermer ce travail, je tiens à remercier les personnes qui m'ont accompagné tout au long de ce parcours.

Ma reconnaissance va d'abord à Monsieur DIDIER BELHOMME, chargé de cours responsable de l'unité d'enseignement « Épreuve intégrée », pour son encadrement méthodologique, la clarté de ses consignes et l'exigence avec laquelle il m'a appris à structurer et à rédiger ce travail.

Je remercie également Monsieur VINCENT WILMET pour son accompagnement technique et sa disponibilité. Nos échanges ont nourri plusieurs choix d'architecture et m'ont souvent aidé à prendre du recul sur le code.

Mes remerciements s'adressent aussi à l'ensemble des enseignants de l'Institut Saint-Laurent, qui m'ont transmis au fil de ces années les compétences que ce projet tente de mettre en pratique.

Je pense enfin à mes collègues de CINAF, avec qui j'ai partagé le contexte de mon stage et qui ont rendu ce projet concret, et à ma famille ainsi qu'à mes proches, pour leur patience et leur soutien de chaque jour.

Table des matières

Introduction.....	1
PARTIE I — ANALYSE.....	2
1. Présentation du projet et cahier des charges.....	3
1.1. Le besoin métier.....	3
1.2. Du prototype monolithique à la version 2.....	3
1.3. Périmètre retenu.....	3
2. Les acteurs du système.....	4
3. Cas d'utilisation.....	5
4. Modèle de données.....	6
4.1. Le catalogue.....	6
4.2. Comptes, studios et abonnements.....	7
4.3. Le schéma relationnel.....	9
5. Processus métiers.....	11
5.1. Publier une œuvre.....	11
5.2. Du visiteur à la lecture.....	12
5.3. Retirer une œuvre.....	14
6. Exigences fonctionnelles et non fonctionnelles.....	15
6.1. Exigences fonctionnelles.....	15
6.2. Exigences non fonctionnelles.....	15
6.3. Contraintes.....	15
PARTIE II — CONTEXTE TECHNOLOGIQUE.....	16
7. Architecture globale.....	17
7.1. Une architecture découplée.....	17
7.2. Un monolithe modulaire.....	17
7.3. Hiérarchie des rôles et séparation des pouvoirs.....	18
7.4. Hébergement et déploiement.....	18
8. Pile technique et justification des choix.....	19
8.1. Symfony et API Platform.....	20
8.2. PostgreSQL plutôt que MySQL.....	20
8.3. Next.js et React.....	21
8.4. Technologies envisagées puis écartées.....	21
9. Sécurité : une défense en profondeur.....	21
10. Intégrations tierces.....	22
10.1. Bunny.net.....	22
10.2. Stripe.....	22

PARTIE III — RÉALISATION TECHNIQUE.....	23
11. Méthodologie de développement.....	24
12. Sprint 1 — Authentification et utilisateurs.....	24
13. Sprint 2 — Catalogue et diffusion vidéo.....	25
13.1. Un catalogue, deux sources.....	25
13.2. Importer le catalogue réel.....	26
13.3. Les affiches : relier ce qui n’a pas de clé.....	26
13.4. Côté interface.....	27
14. Sprint 3 — Abonnement et paiement.....	27
15. Sprint 4 — Module des studios de production.....	28
16. Sprint 5 — Lecteur vidéo et diffusion.....	29
17. Sprint 7 — Administration de la plateforme.....	30
18. Organisation du serveur et documentation de l’API.....	30
19. L’interface utilisateur.....	32
20. Tests automatisés.....	33
21. Difficultés rencontrées.....	34
21.1. Un environnement de développement exigeant.....	34
21.2. Des performances difficiles à mesurer.....	34
21.3. Un chantier abandonné : le transcodage par Bunny Stream.....	34
Conclusion.....	35
Bibliographie.....	37
Annexes.....	i
Annexe A — Le cahier des charges et sa mise en œuvre.....	ii
Annexe B — Le dossier d’analyse.....	iv
Annexe C — Dictionnaire des données.....	v
Annexe D — Livrables électroniques.....	viii
Annexe E — Lexique.....	ix

Table des figures

Figure 1 — Diagramme de cas d'utilisation de CINAF v2.....	5
Figure 2 — Diagramme de classes : le catalogue.....	7
Figure 3 — Diagramme de classes : comptes, studios et abonnements.....	8
Figure 4 — Schéma relationnel de la base de données (clés et colonnes principales).....	10
Figure 5 — Processus 1 : publication d'une œuvre par un studio.....	12
Figure 6 — Processus 2 : du visiteur à la lecture d'une œuvre.....	13
Figure 7 — Processus 3 : retrait d'une œuvre du catalogue.....	14
Figure 8 — Diagramme de paquetages de l'application.....	18
Figure 9 — Architecture de déploiement et flux entre les composants.....	19
Figure 10 — Les onze contrôleurs du contexte public et utilisateur.....	31
Figure 11 — Arborescence des trente-six pages de l'interface.....	33

Introduction

Ce travail de fin d'études présente CINAF v2, une plateforme web de streaming consacrée au cinéma africain indépendant. Un spectateur peut y découvrir des films et des séries puis les regarder grâce à un abonnement ; un studio de production peut y publier lui-même ses œuvres, sous le contrôle d'un administrateur. Pour un futur informaticien de gestion, le sujet a l'intérêt de réunir presque tout ce qui fait une application professionnelle : des données métier structurées, des droits d'accès à plusieurs niveaux, du paiement en ligne, de la vidéo diffusée à grande échelle et des services tiers qu'il faut faire travailler ensemble.

Le projet s'inscrit dans un contexte réel. Il prolonge le stage que j'ai effectué chez CINAF, entreprise dans laquelle je reste impliqué, et repart d'un premier prototype dont il propose une refonte complète. Le catalogue de démonstration n'a d'ailleurs pas été inventé : il a été reconstitué à partir des vidéos que l'entreprise héberge réellement chez Bunny.net, avec les surprises que cela suppose et que la troisième partie raconte.

Le rapport suit les trois temps proposés par le guide de l'épreuve intégrée. La première partie est consacrée à l'analyse : le besoin, les acteurs, les cas d'utilisation, le modèle de données et les processus métiers, illustrés par des diagrammes UML et BPMN redessinés pour correspondre à l'application telle qu'elle fonctionne aujourd'hui. La deuxième partie décrit le contexte technologique : l'architecture, la pile technique, les choix argumentés (y compris ceux qui ont été écartés), la sécurité et les intégrations avec Bunny.net et Stripe. La troisième partie retrace la réalisation, sprint après sprint, en s'attardant sur les difficultés rencontrées et sur la manière dont elles ont été comprises puis contournées. La conclusion fait le bilan, assume les limites de la version livrée et ouvre quelques pistes pour la suite.

PARTIE I — ANALYSE

Cette première partie pose le cadre du projet. Elle reprend l'essentiel du cahier des charges et du dossier d'analyse, tous deux fournis en annexe, et les confronte à ce qui a réellement été construit. Les diagrammes qui l'accompagnent ont été redessinés à partir du code : ils décrivent l'application livrée, et non une intention de départ.

1. Présentation du projet et cahier des charges

1.1. Le besoin métier

CINAF v2 part d'un constat simple : le cinéma africain manque d'une vitrine numérique à la hauteur de sa production. Les grandes plateformes internationales n'en proposent qu'une petite part, souvent limitée aux productions anglophones les mieux financées, tandis que les créateurs indépendants peinent à trouver un canal de diffusion qui leur laisse la main sur leurs œuvres. La plateforme veut réunir ces deux mondes dans un même espace.

La proposition de valeur se décline selon trois publics. Le spectateur découvre des œuvres africaines en haute définition, avec un abonnement unique qui ouvre l'ensemble du catalogue. Le studio de production dispose d'un espace en libre-service pour mettre ses contenus en ligne, suivre leur statut et, à terme, être rémunéré selon l'audience. L'administrateur, enfin, dispose des outils de modération, de gestion des comptes et de supervision du catalogue sans lesquels l'ensemble ne tiendrait pas longtemps.

1.2. Du prototype monolithique à la version 2

La première version reposait sur un monolithe classique dont les pages étaient entièrement produites par le serveur. Elle a vite montré ses limites : impossible d'envisager une application mobile sans tout réécrire, rechargement complet de la page à chaque action et logique métier dispersée entre le code serveur et les gabarits. La version 2 sépare donc nettement l'interface, écrite avec Next.js, et une API REST écrite avec Symfony. Les conséquences de ce choix sont détaillées dans la deuxième partie.

1.3. Périmètre retenu

Le périmètre a été pensé pour démontrer une chaîne de valeur complète : un visiteur crée un compte, souscrit un abonnement, parcourt le catalogue et lance la lecture d'un film ; en parallèle, un membre devient producteur, met en ligne son premier contenu et le soumet à l'administrateur avant qu'il soit visible. Les modules d'authentification, de catalogue, de lecture vidéo, d'abonnement et de paiement, de gestion des studios et d'administration font partie de ce périmètre.

Certains volets du cahier des charges ont été écartés volontairement pour concentrer l'effort sur ce cœur fonctionnel : l'application mobile native, la recommandation algorithmique, la rémunération détaillée des studios, les sous-titres et le multilinguisme de l'interface. La notation et les commentaires, prévus au départ, ont été reportés. Une offre d'achat à l'unité et plusieurs

paliers d'abonnement avaient aussi été envisagés ; ils ont été remplacés par un abonnement unique, proposé au mois ou à l'année, plus simple à expliquer et à maîtriser. L'annexe A met en regard, exigence par exigence, ce que prévoyait le cahier des charges et ce que l'application fait réellement.

2. Les acteurs du système

L'analyse distingue quatre acteurs principaux, dont les droits s'emboîtent : chacun hérite de ce que peut faire le précédent, à une exception près, discutée au chapitre 7. Deux services externes interviennent en outre comme acteurs secondaires : Stripe pour le paiement et Bunny.net pour le stockage et la diffusion des vidéos.

Acteur	Rôle dans l'application	Rôle technique
Visiteur	Parcourt le catalogue, consulte les fiches et les pages des studios, regarde les bandes-annonces, crée un compte ou se connecte.	aucun (non authentifié)
Utilisateur inscrit	Gère son profil et ses données personnelles, souscrit un abonnement et, tant qu'il est actif, regarde films et épisodes ; peut suivre gratuitement des studios et demander à devenir producteur.	ROLE_USER
Studio producteur	Gère son studio, crée les fiches de ses films et séries, téléverse vidéos, affiches et bandes-annonces, demande la publication ou le retrait de ses œuvres.	ROLE_CREATEUR
Administrateur	Gère les comptes et les abonnements des membres, approuve le premier contenu des nouveaux studios, instruit les demandes de retrait, supervise le catalogue et le stockage vidéo.	ROLE_ADMIN

Tableau 1 — Les acteurs de CINAF v2

Un point mérite d'être relevé dès l'analyse : être « abonné » n'est pas un rôle, c'est un état. Le cahier des charges prévoyait un rôle dédié ; dans l'application, le droit de regarder une vidéo dépend uniquement de l'existence, en base de données, d'un abonnement actif et non expiré. Ce choix évite qu'un rôle oublié dans un compte continue d'ouvrir l'accès après la fin d'un abonnement, et il simplifie la résiliation : il suffit que la date de fin soit dépassée pour que l'accès se ferme.

3. Cas d'utilisation

Les interactions entre les acteurs et le système sont résumées dans le diagramme de cas d'utilisation ci-dessous, établi selon la notation UML (Object Management Group, 2017)¹. Il se lit de haut en bas, par bandes : ce que fait le visiteur, ce que gagne l'utilisateur inscrit, puis les espaces réservés aux studios et à l'administration.

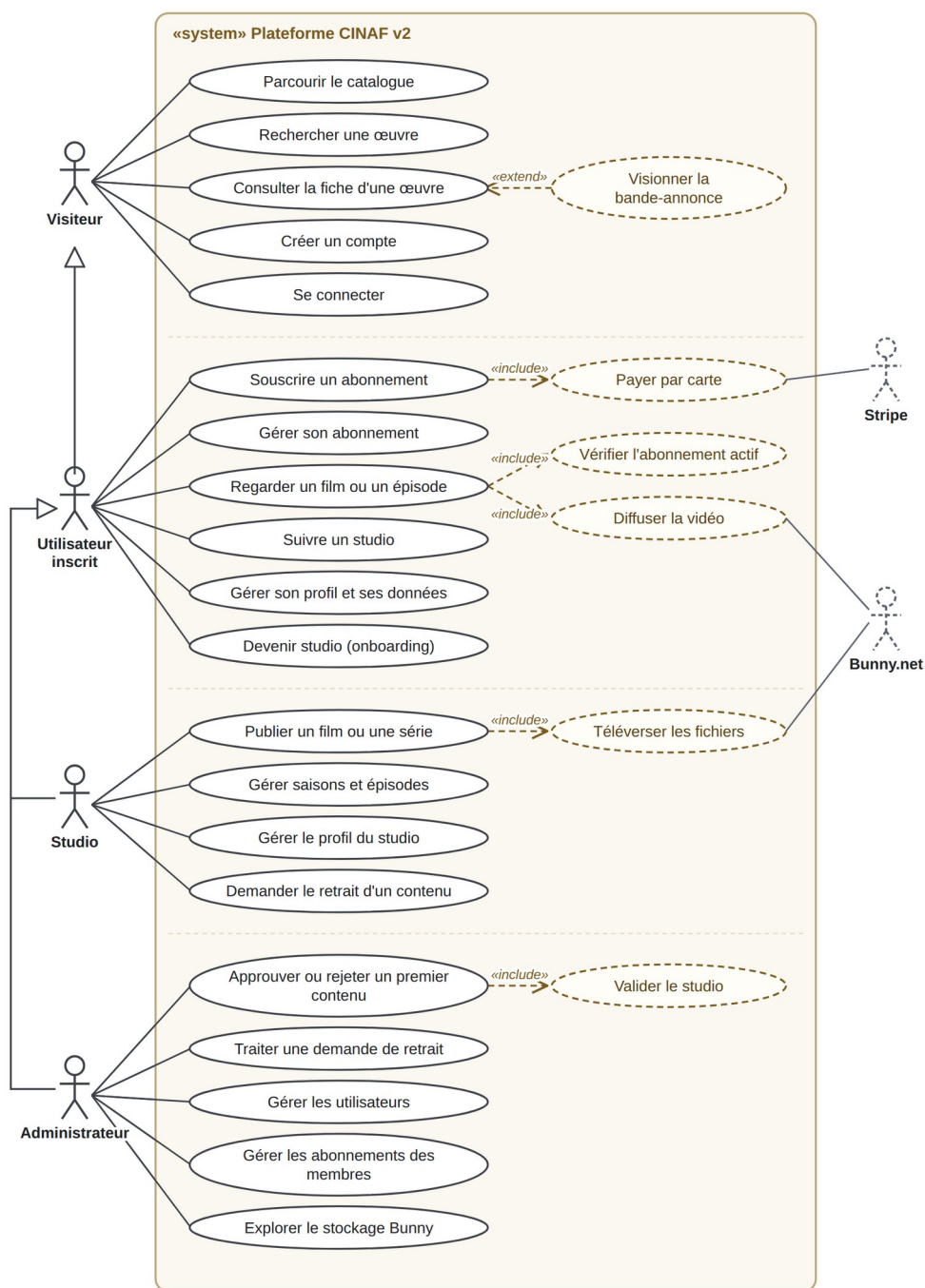


Figure 1 — Diagramme de cas d'utilisation de CINAF v2

¹Les diagrammes de cas d'utilisation détaillés par module figurent dans le dossier d'analyse (annexe B) ; la version présentée ici est une synthèse redessinée à partir de l'application livrée.

Les flèches de généralisation, à gauche, traduisent l'emboîtement des droits : l'utilisateur inscrit peut tout ce que peut le visiteur, et le studio comme l'administrateur sont d'abord des utilisateurs inscrits. Les relations d'inclusion signalent les étapes qui font obligatoirement partie d'un cas : regarder une vidéo inclut toujours la vérification de l'abonnement puis la diffusion par Bunny.net ; souscrire inclut le paiement par carte chez Stripe ; approuver le premier contenu d'un studio inclut la validation de ce studio. La bande-annonce, à l'inverse, est une extension facultative de la consultation d'une fiche.

Ce découpage a servi de fil conducteur pour la suite : il structure les processus métiers présentés plus loin, l'organisation du code en trois contextes et le découpage du développement en sprints.

4. Modèle de données

Le modèle de données est le socle de l'application. Il compte dix-sept entités Doctrine, traduites en vingt-six tables PostgreSQL. Plutôt qu'un diagramme global illisible, il est présenté ici en deux vues complémentaires : le catalogue d'une part, les comptes, studios et abonnements d'autre part. Le schéma relationnel vient ensuite montrer comment ces classes deviennent des tables.

4.1. Le catalogue

Le catalogue distingue deux familles d'œuvres. Un film peut être découpé en plusieurs parties, ce qui reflète la réalité du catalogue importé, où certains longs métrages sont stockés en plusieurs fichiers. Une série se compose de saisons, elles-mêmes composées d'épisodes : ce sont des compositions au sens UML, car une saison n'a pas d'existence hors de sa série. Films et séries partagent des référentiels communs (genres, pays, personnes et étiquettes), reliés par des associations plusieurs-à-plusieurs.

Dans le diagramme de la figure 2, deux classes méritent un mot. `FeaturedContent` sert à mettre une œuvre en avant sur la page d'accueil pendant une période donnée. `Language`, en revanche, existe dans le modèle mais n'est reliée à rien : elle avait été prévue pour gérer la langue des œuvres et n'a jamais été branchée. Plutôt que de la masquer, je l'ai laissée visible et signalée comme une dette technique. Enfin, les constantes de statut portées par `Film` et `Serie` (brouillon, en attente d'approbation, publié, retiré) encodent tout le cycle de vie d'une œuvre ; elles reviennent dans les processus métiers.

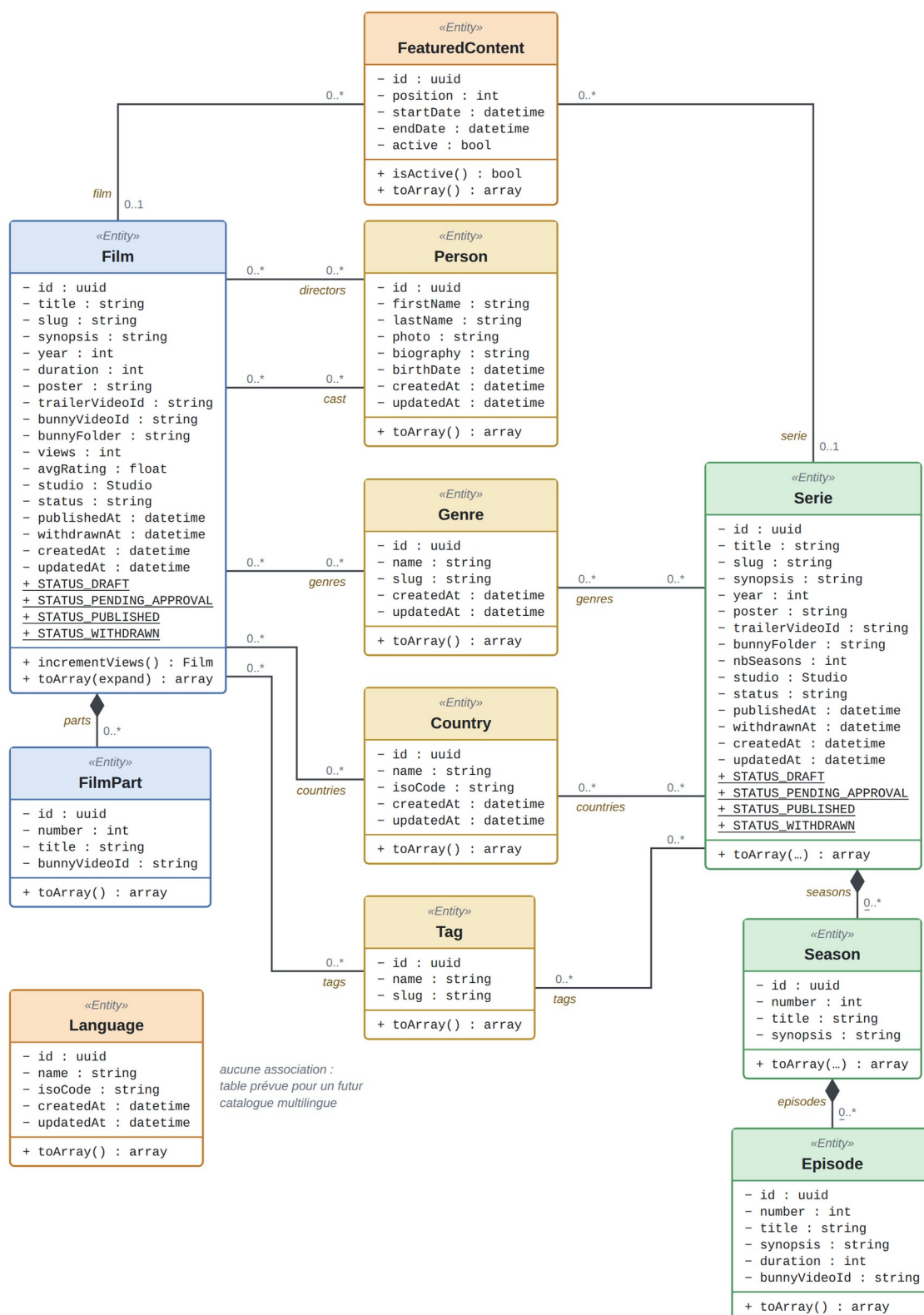


Figure 2 — Diagramme de classes : le catalogue

4.2. Comptes, studios et abonnements

La seconde vue regroupe tout ce qui concerne les personnes et l'argent. User implémente les deux interfaces de sécurité de Symfony, ce qui permet au composant d'authentification de

l'utiliser directement. Un utilisateur possède au plus un studio : la relation est de type « un à un », garantie en base par une contrainte d'unicité sur le propriétaire. Le studio porte un indicateur `isValidated` qui passe à vrai lorsque l'administrateur approuve son premier contenu.

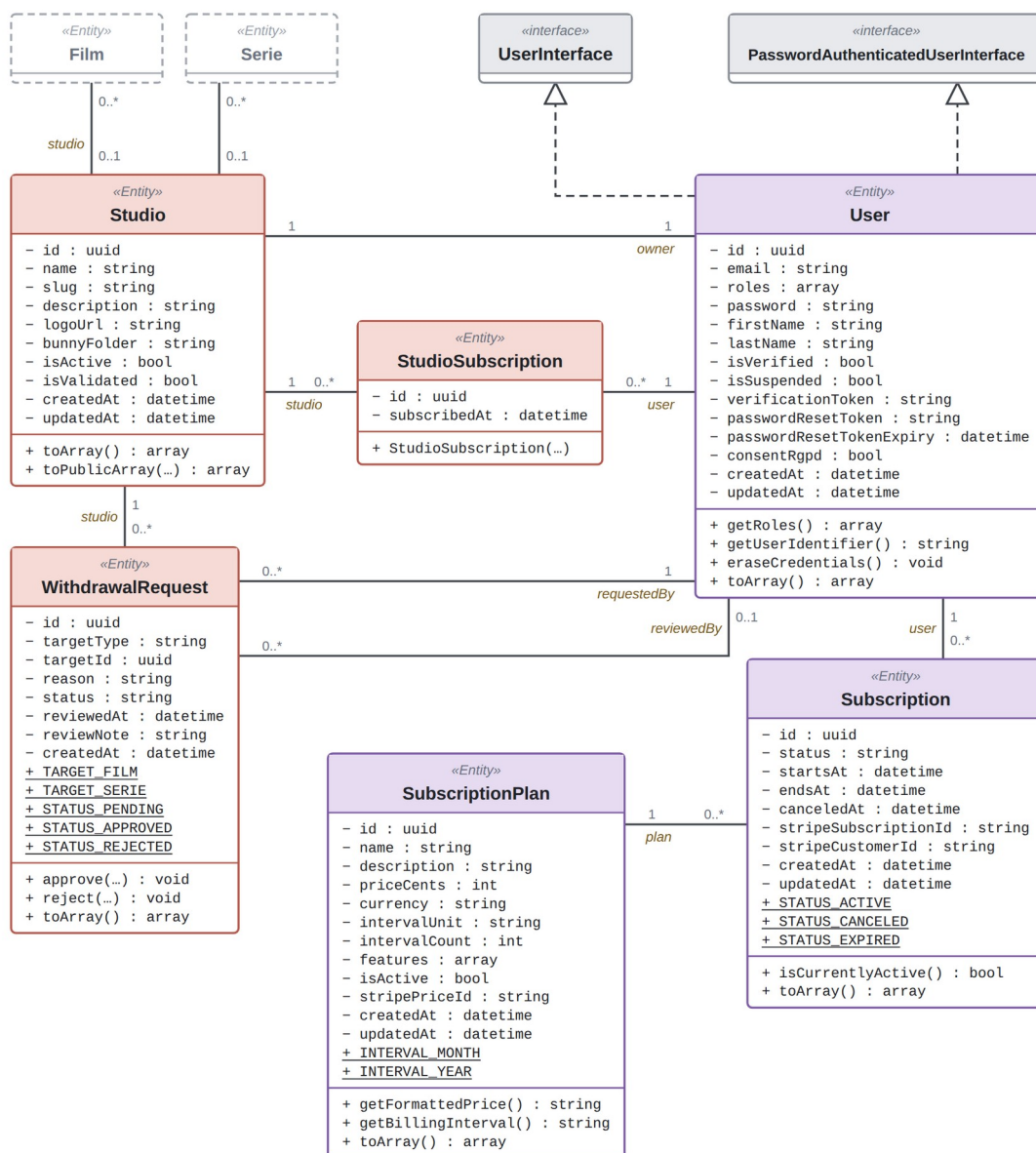


Figure 3 — Diagramme de classes : comptes, studios et abonnements

Deux notions d'abonnement coexistent volontairement et ne doivent pas être confondues. `Subscription`, rattachée à une formule `SubscriptionPlan`, est l'abonnement payant qui ouvre la lecture. `StudioSubscription` est un simple suivi gratuit, comparable à l'abonnement à une chaîne sur une plateforme de vidéos : il alimente le compteur d'abonnés affiché sur la page du studio. Quant à `WithdrawalRequest`, la demande de retrait, elle vise soit un film, soit une

série. Cette cible est dite polymorphe : elle est enregistrée sous la forme d'un type et d'un identifiant plutôt que par une clé étrangère, d'où les classes en pointillés sur le diagramme.

4.3. Le schéma relationnel

Le schéma de la figure 4, présenté en pleine page, montre le passage du modèle objet aux tables. Pour rester lisible à l'impression, il ne présente que les clés et les colonnes les plus parlantes ; la description complète des vingt-six tables, avec leurs types et leurs contraintes, figure dans le dictionnaire des données de l'annexe C.

Quelques choix de conception ressortent. Toutes les clés primaires sont des UUID, générés par l'application : ils ne révèlent pas le volume de données et peuvent être créés avant même l'écriture en base. Les associations plusieurs-à-plusieurs passent par des tables de jointure dont la clé primaire est le couple des deux clés étrangères. Plusieurs règles métier sont confiées directement à la base, là où elles ne peuvent pas être contournées : un numéro d'épisode unique dans sa saison, un seul studio par propriétaire, un seul suivi par couple utilisateur-studio, et surtout une seule demande de retrait en attente par œuvre, grâce à un index unique partiel que PostgreSQL sait exprimer.

Le schéma garde aussi la trace de ce qui n'est pas parfait. La table `refresh_tokens` appartient au module de rafraîchissement des jetons et ne référence l'utilisateur que par son adresse, sans clé étrangère. La table `language`, déjà évoquée, reste isolée. Ces écarts sont connus ; ils n'empêchent rien de fonctionner, mais un lecteur attentif du schéma les verra, et il vaut mieux qu'il les trouve expliqués.

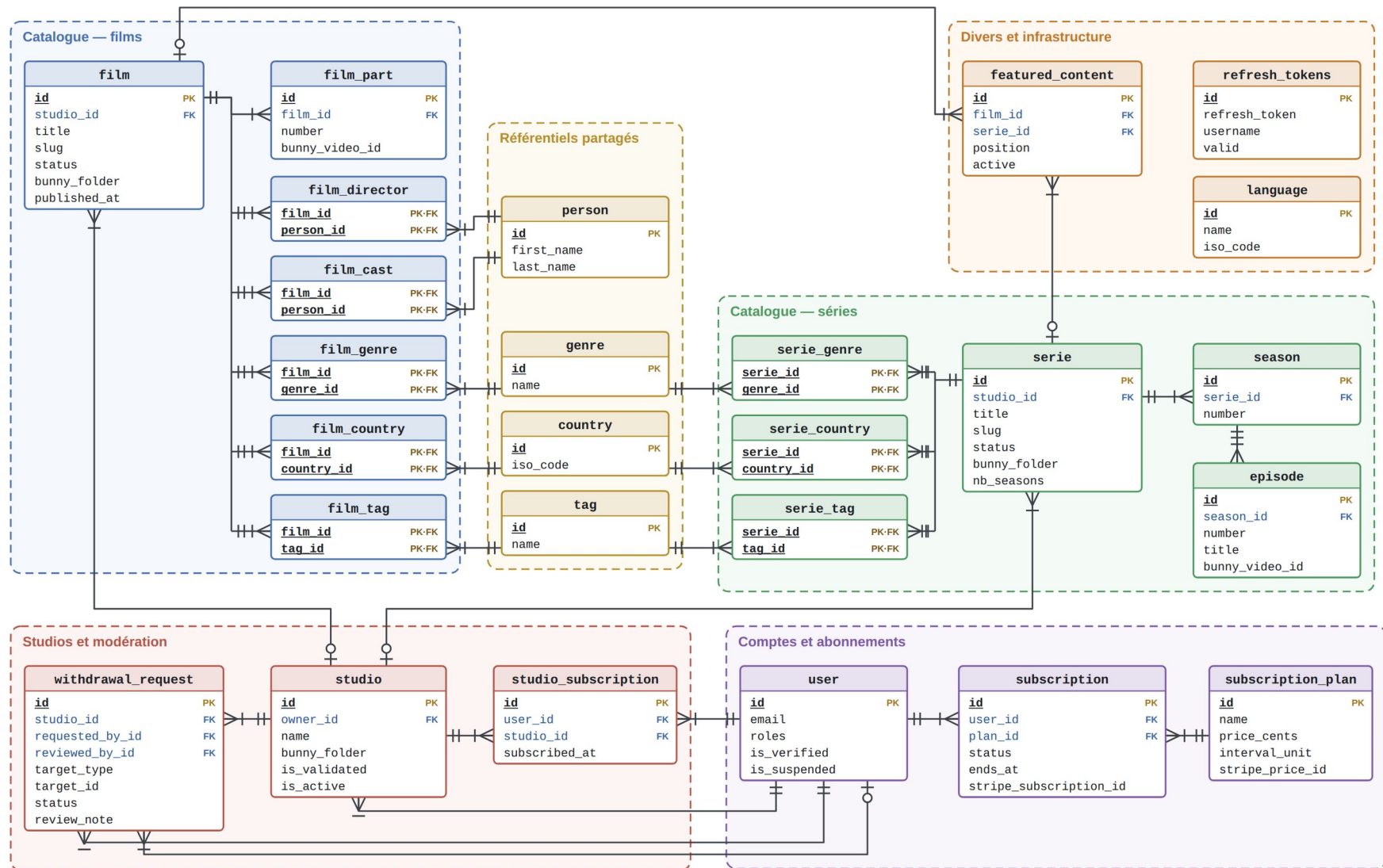


Figure 4 — Schéma relationnel de la base de données (clés et colonnes principales)

5. Processus métiers

Le dossier d'analyse décrivait vingt-huit processus métiers détaillés, répartis en sept modules. Trois d'entre eux portent l'essentiel de la valeur de la plateforme et ont été redessinés en BPMN² pour coller exactement au comportement du code : la publication d'une œuvre par un studio, le parcours qui mène un visiteur jusqu'à la lecture, et le retrait d'une œuvre. Chaque diagramme se lit de haut en bas ; les couloirs indiquent qui agit, les tâches marquées d'un engrenage sont exécutées par le système, celles marquées d'un personnage par un humain.

5.1. Publier une œuvre

Le premier processus (figure 5) commence par un membre connecté qui choisit de devenir producteur. Son studio est créé non validé et il reçoit le rôle de créateur. Il crée ensuite la fiche de son œuvre, toujours en brouillon, puis téléverse la vidéo, l'affiche et la bande-annonce, que la plateforme range dans son dossier sur Bunny Storage. Au moment de la demande de publication, une seule question décide du chemin : le studio est-il déjà validé ? Si oui, l'œuvre est publiée immédiatement. Sinon, elle passe en attente et rejoint la file d'approbation de l'administrateur. Celui-ci examine à la fois l'œuvre et le studio ; en approuvant, il publie l'œuvre et valide le studio d'un même geste, si bien que les contenus suivants seront publiés sans délai. En rejetant, il renvoie l'œuvre en brouillon.

Ce processus comporte aussi deux limites assumées : le motif d'un rejet n'est pas conservé, et aucun courriel n'avertit le producteur. Celui-ci découvre la décision en consultant le statut de son œuvre dans son espace studio.

²Business Process Model and Notation, norme de modélisation des processus publiée par l'Object Management Group (Object Management Group, 2011).

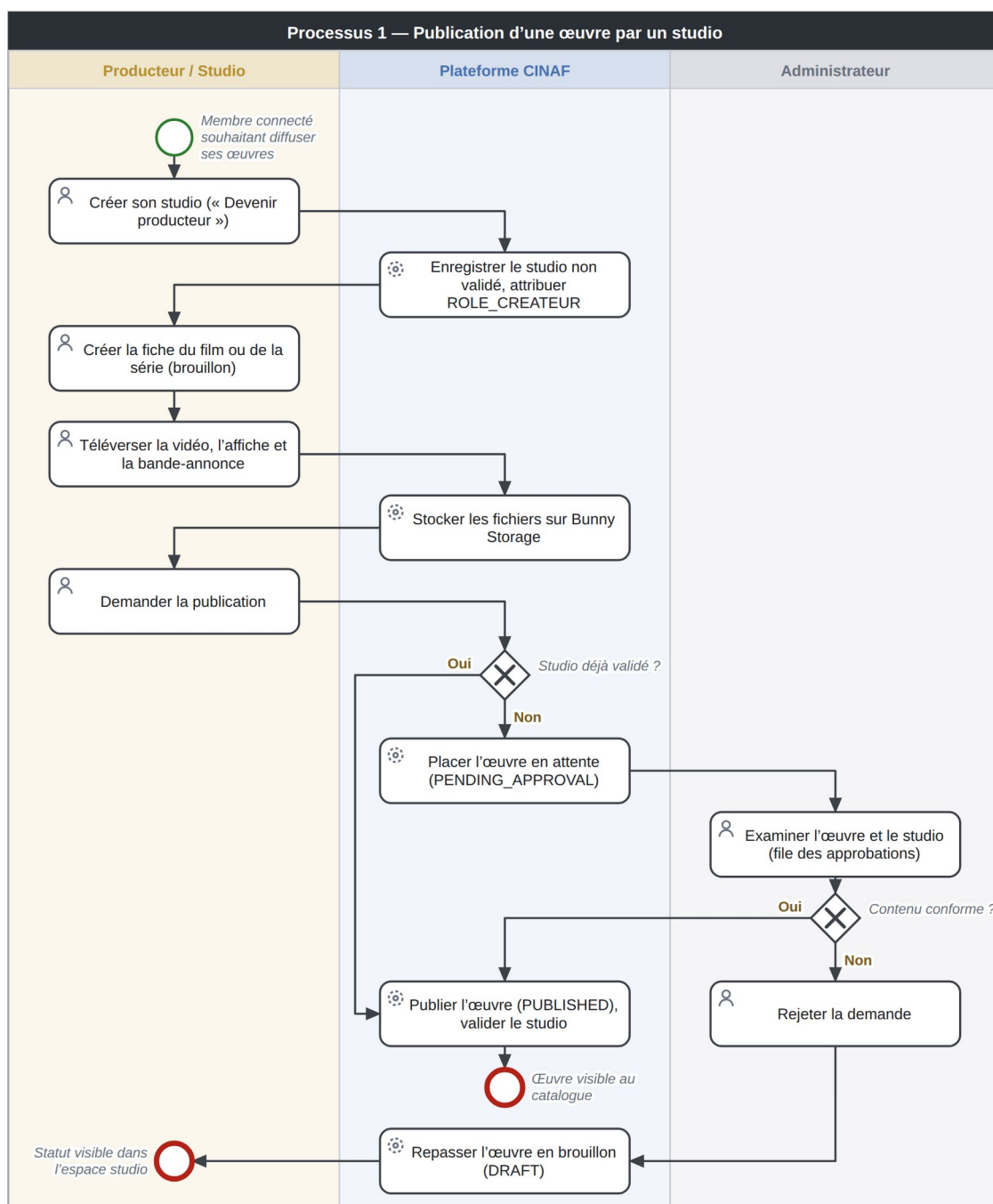


Figure 5 — Processus 1 : publication d'une œuvre par un studio

5.2. Du visiteur à la lecture

Le deuxième processus (figure 6) suit le spectateur. La création d'un compte ne le connecte pas : la plateforme enregistre le compte et prépare un lien de vérification, puis l'utilisateur se connecte et reçoit un jeton d'accès valable quinze minutes, accompagné d'un jeton de rafraîchissement. Lorsqu'il lance une lecture, la plateforme répond à une seule question, « cet utilisateur a-t-il un abonnement actif ? ». S'il en a un, le chemin court, à droite, mène directement à la vidéo. S'il n'en a pas, il choisit une formule, à 9,99 € par mois ou 99 € par an, paie par carte dans un formulaire Stripe intégré à la page, et son abonnement est activé.

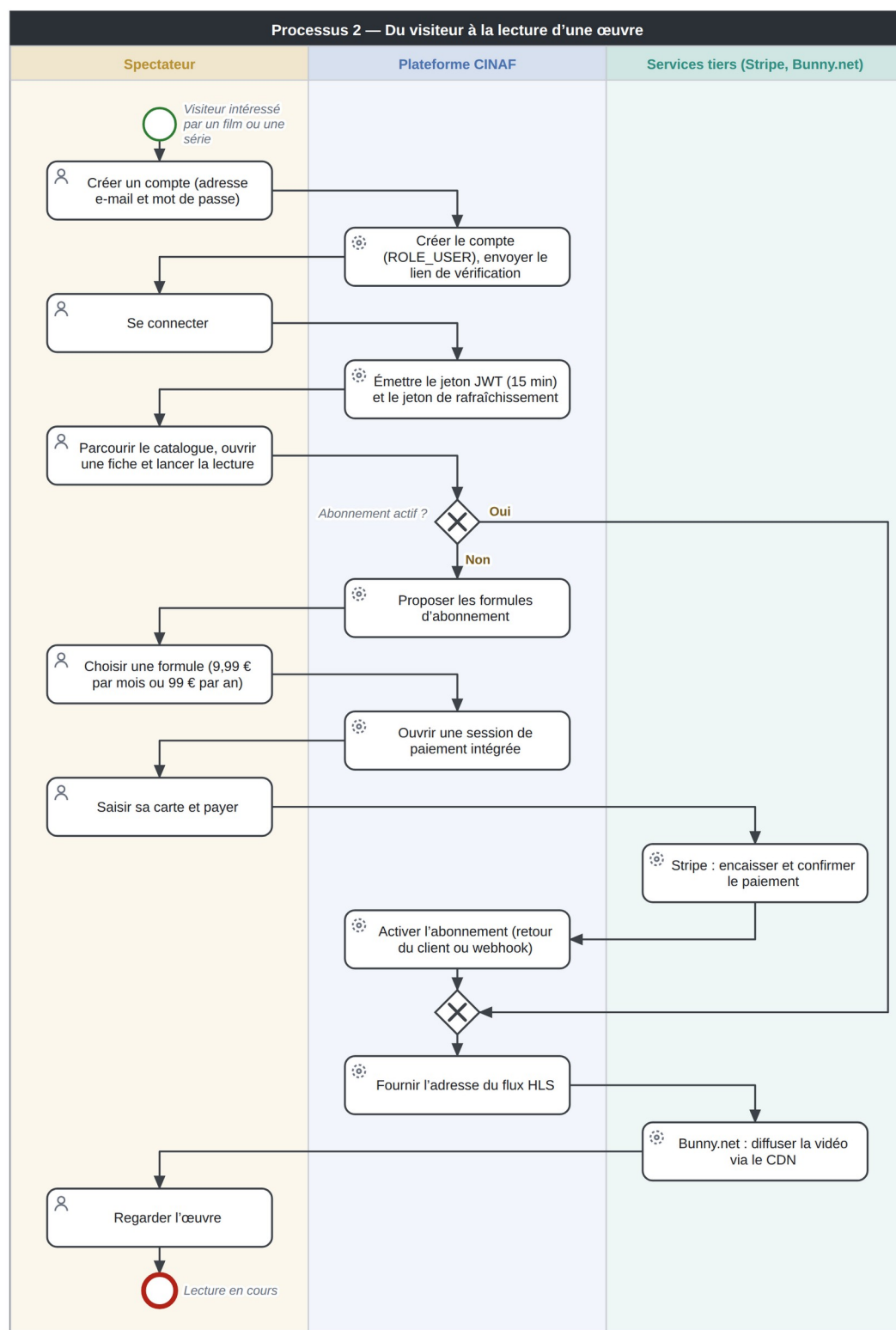


Figure 6 — Processus 2 : du visiteur à la lecture d'une œuvre

L'activation mérite une remarque, car elle a fait l'objet d'une correction importante, racontée au chapitre 14 : elle est déclenchée à la fois au retour du client sur la page de confirmation et par la notification envoyée par Stripe. Les deux chemins aboutissent au même

résultat sans jamais créer de doublon. Une fois le droit établi, la plateforme fournit l'adresse du flux vidéo et c'est le réseau de diffusion de Bunny.net qui sert la vidéo au navigateur.

5.3. Retirer une œuvre

Le troisième processus (figure 7) part d'un constat simple. Une œuvre publiée ne peut pas être supprimée d'un clic par son studio : elle a pu être regardée, mise en avant ou suivie. Le studio doit donc en demander le retrait en motivant sa demande. La plateforme refuse une seconde demande tant que la première est en attente, règle garantie par l'index partiel évoqué plus haut. L'administrateur instruit ensuite la demande. S'il l'accepte, l'œuvre passe au statut « retiré » : elle disparaît du catalogue et sa fiche répond désormais par une page dédiée indiquant que le contenu n'est plus disponible, plutôt que par une simple erreur. S'il la refuse, l'œuvre reste publiée et le motif du refus est conservé avec la demande.

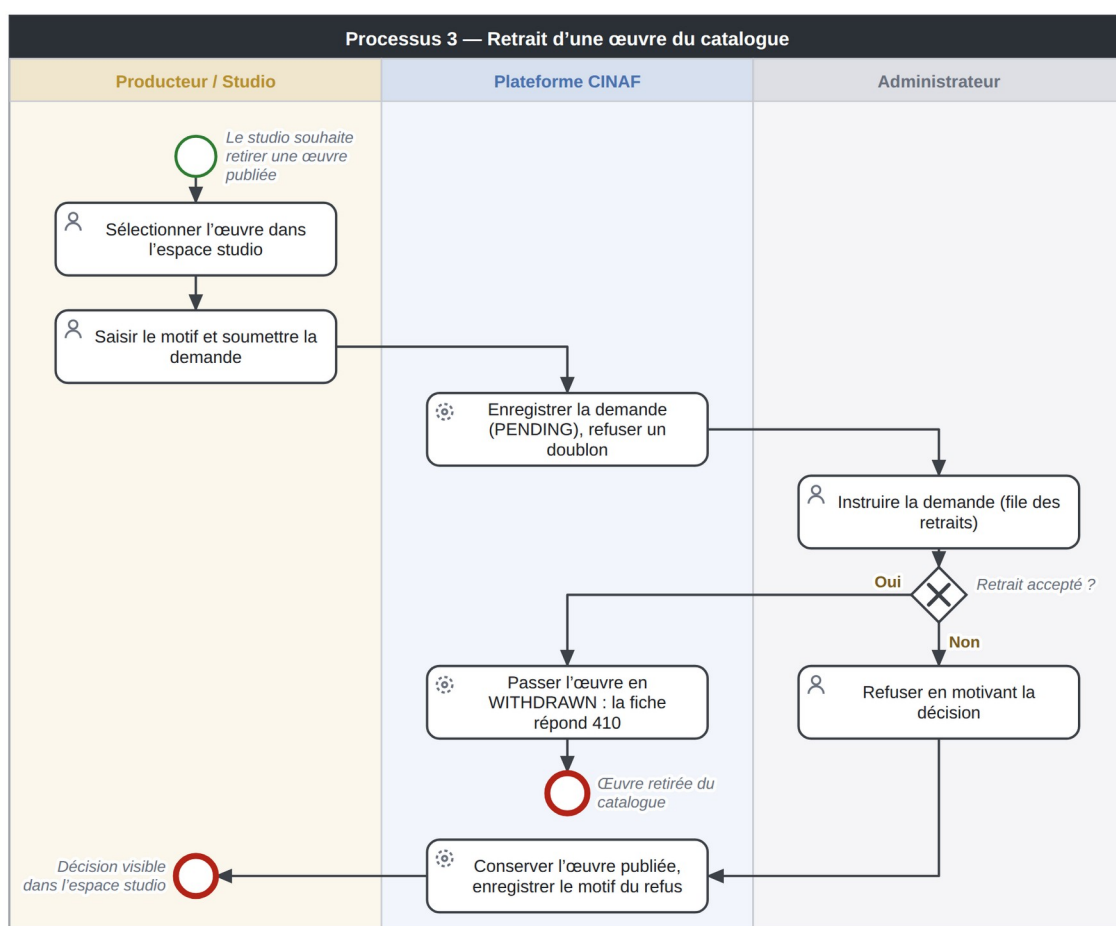


Figure 7 — Processus 3 : retrait d'une œuvre du catalogue

6. Exigences fonctionnelles et non fonctionnelles

6.1. Exigences fonctionnelles

Les exigences fonctionnelles se répartissent selon les modules du cas d'utilisation :

- authentification : inscription avec lien de vérification, connexion, rafraîchissement et déconnexion, réinitialisation du mot de passe, gestion du profil, export et suppression des données personnelles ;
- catalogue : navigation par listes et carrousels, recherche par titre, fiches détaillées des films et des séries, pages publiques des studios ;
- abonnement et paiement : choix d'une formule, paiement par carte, résiliation différée et réactivation, historique des paiements ;
- lecture : contrôle du droit de lecture, lecture adaptative en HLS, bandes-annonces accessibles à tous ;
- studios : création du studio, gestion des fiches, téléversement des fichiers, saisons et épisodes, demandes de publication et de retrait ;
- administration : comptes et rôles, abonnements des membres, file d'approbation, demandes de retrait, supervision du catalogue et du stockage vidéo.

6.2. Exigences non fonctionnelles

Au-delà des fonctionnalités, la plateforme devait satisfaire des exigences de qualité : sécurité des données et des accès, fluidité de la navigation, maintenabilité du code, compatibilité avec les navigateurs courants et conformité au Règlement général sur la protection des données (Union européenne, 2016). Cette dernière s'est traduite concrètement par un consentement recueilli à l'inscription et par deux fonctions offertes à chaque utilisateur : l'export de ses données et la suppression de son compte. Le cahier des charges fixait aussi des objectifs chiffrés, comme un temps de réponse de l'API inférieur à 500 ms pour 95 % des requêtes ; la section consacrée aux performances, au chapitre 21, explique pourquoi cet objectif n'a pas pu être mesuré honnêtement en développement.

6.3. Contraintes

Le projet a été mené dans un cadre contraint : un poste de développement sous Windows, plusieurs services tiers à intégrer, les obligations légales du RGPD et la réalité d'un travail réalisé seul, sur une durée limitée et avec un budget réduit. Ces contraintes ont pesé sur plusieurs arbitrages, à commencer par le choix d'un monolithe modulaire plutôt que de microservices.

PARTIE II — CONTEXTE TECHNOLOGIQUE

Cette partie décrit l'environnement technique dans lequel CINAF v2 a été construit. L'objectif n'est pas d'aligner des noms de technologies, mais d'expliquer pourquoi elles ont été retenues, parfois contre l'avis du cahier des charges initial, et quelles alternatives ont été écartées.

7. Architecture globale

7.1. Une architecture découplée

Le cœur de l'architecture repose sur la séparation entre l'interface utilisateur et le serveur. Le serveur, écrit avec Symfony, ne produit plus de pages HTML : il expose une API REST qui reçoit et renvoie du JSON. L'interface, écrite avec Next.js et React, consomme cette API depuis le navigateur et se charge de toute la présentation. L'utilisateur est identifié par un jeton JWT (JONES, BRADLEY & SAKIMURA, 2015) joint à chaque requête, ce qui dispense le serveur de conserver une session.

Ce découplage a un coût : deux bases de code, deux chaînes d'outils, deux conteneurs à déployer. Il ouvre en revanche des perspectives décisives pour un projet qui vise une mise en production. Une application mobile pourra réutiliser la même API sans modification, un partenaire pourra s'y connecter, et chaque couche pourra évoluer à son rythme.

7.2. Un monolithe modulaire

Côté serveur, l'application est un monolithe : un seul déploiement, une seule base de données et une suite de tests exécutable de bout en bout. Avec une équipe d'une personne, la complexité opérationnelle des microservices (orchestration, communication entre services, supervision distribuée) aurait été disproportionnée. Le monolithe n'est pas pour autant un bloc indifférencié. Le code est organisé en trois contextes délimités, chacun avec ses contrôleurs : le contexte public et utilisateur, l'espace des studios et l'administration. Ils partagent les entités, les dépôts de données et les services transverses.

Le diagramme de paquetages (figure 8) fait apparaître une seule dépendance entre contextes, tracée en doré : les contrôleurs d'administration utilisent le service du contexte studio qui gère le cycle de vie des contenus. C'est voulu. Lorsqu'un administrateur approuve un premier contenu ou accepte un retrait, il doit appliquer exactement les mêmes règles de transition que le studio ; dupliquer ces règles dans l'administration aurait été la meilleure façon de les voir diverger. Cette discipline garde le code lisible et permettrait, si la charge le justifiait un jour, d'extraire un contexte en service autonome.

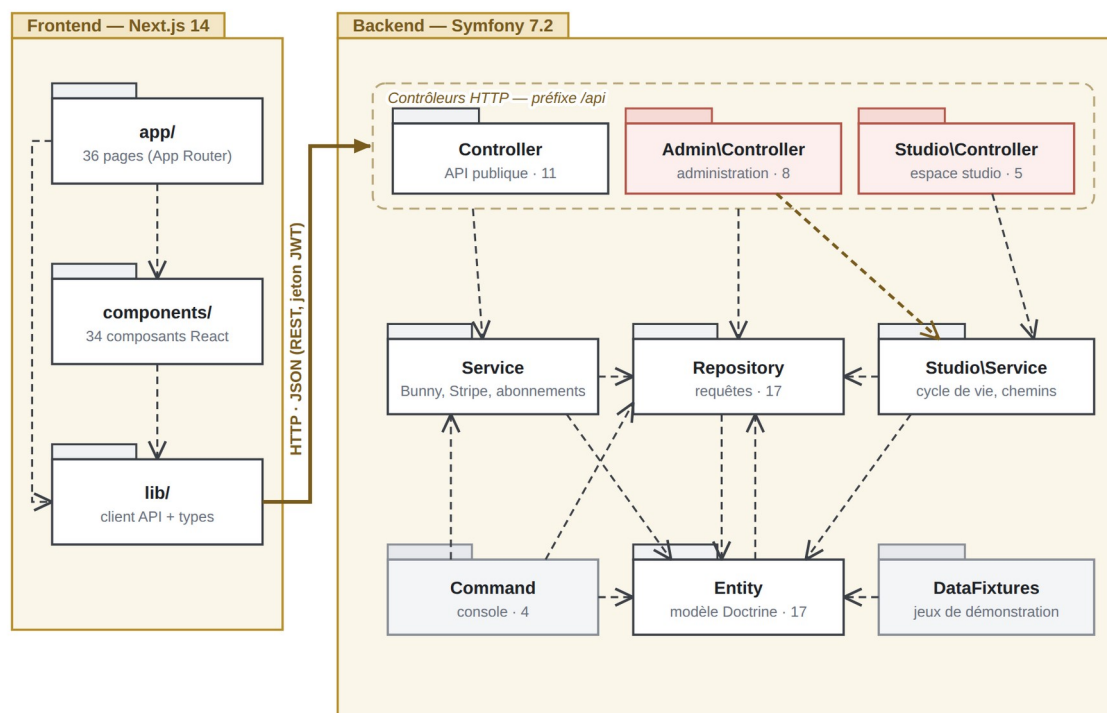


Figure 8 — Diagramme de paquetages de l'application

7.3. Hiérarchie des rôles et séparation des pouvoirs

Les droits reposent sur une hiérarchie de rôles déclarée dans la configuration de sécurité. Le rôle d'abonné, le rôle de créateur et le rôle de modérateur héritent tous du rôle d'utilisateur ; l'administrateur hérite du modérateur, de l'abonné et de l'utilisateur. Il n'hérite volontairement pas du rôle de créateur. Si c'était le cas, un administrateur pourrait utiliser les points d'entrée des studios, se créer un studio fictif et publier ses propres contenus sans passer par la modération qu'il est censé exercer. Cette séparation des pouvoirs est une décision d'architecture, pas un détail de configuration.

Comme indiqué dans l'analyse, le droit de lecture ne repose pas sur un rôle mais sur l'existence d'un abonnement actif. Le rôle d'abonné subsiste dans la hiérarchie pour des raisons historiques ; il n'ouvre rien à lui seul.

7.4. Hébergement et déploiement

L'application est conditionnée en trois conteneurs Docker, décrits dans un fichier Docker Compose et destinés à un serveur privé virtuel chez OVH. Le conteneur *frontend* exécute Next.js. Le conteneur *backend* exécute Symfony avec FrankenPHP, un serveur d'applications PHP moderne qui remplace le couple traditionnel Nginx et PHP-FPM prévu dans le cahier des charges. Devant eux, Caddy reçoit tout le trafic, obtient et renouvelle automatiquement les

certificats HTTPS auprès de Let’s Encrypt, et oriente chaque domaine vers le bon conteneur (Caddy, 2025). La base PostgreSQL, elle, est confiée à une offre managée, qui prend en charge sauvegardes et mises à jour.

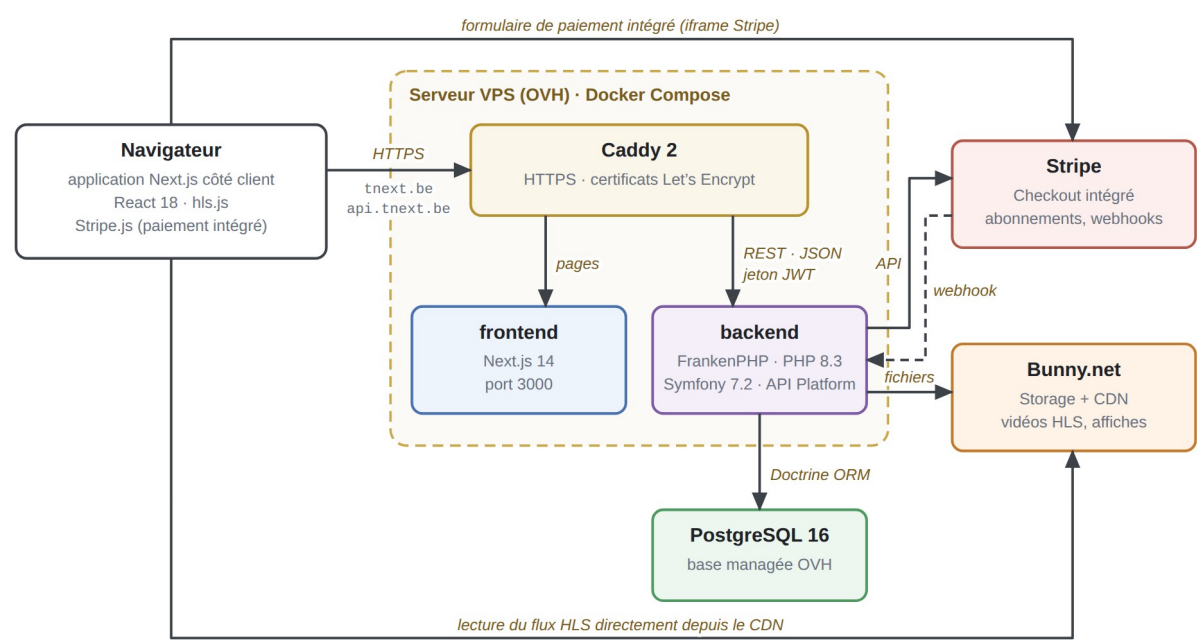


Figure 9 — Architecture de déploiement et flux entre les composants

La figure montre un détail important pour les performances : la vidéo ne transite jamais par le serveur de CINAF. Le navigateur obtient l’adresse du flux auprès de l’API, puis lit les segments directement sur le réseau de diffusion de Bunny.net. De la même manière, les données de carte bancaire ne passent que par le formulaire Stripe intégré à la page : elles ne touchent ni l’interface ni le serveur de la plateforme.

8. Pile technique et justification des choix

Le tableau suivant récapitule les technologies retenues, dans les versions effectivement utilisées, et la raison de chaque choix.

Couche	Technologie	Motivation
Langage serveur	PHP 8.3	Typage strict, attributs, énumérations, propriétés en lecture seule.
Framework serveur	Symfony 7.2	Maturité, composants découplés, composant de sécurité éprouvé.
Couche API	API Platform 4.1	Documentation OpenAPI, sérialisation et pagination normalisées, contrôleurs sur mesure possibles.
Accès aux données	Doctrine ORM 3.6	Standard de l'écosystème Symfony, migrations versionnées.

Couche	Technologie	Motivation
Authentification	JWT RS256 (LexikJWT, Gesdinet)	Jetons sans état signés par une clé privée ; jeton de rafraîchissement à rotation.
Base de données	PostgreSQL 16	Type JSON natif, index uniques partiels, rigueur transactionnelle.
Interface	Next.js 14, React 18, TypeScript 5	Routage par fichiers, composants typés, optimisations intégrées.
Mise en forme	Bootstrap 5.3	Grille et composants standards, identité visuelle par variables CSS.
Lecture vidéo	hls.js 1.6	Lecture HLS adaptative sur les navigateurs qui ne la gèrent pas nativement.
Vidéo et fichiers	Bunny.net (Storage + CDN)	Coût réduit, stockage et diffusion mondiale, catalogue existant de l'entreprise.
Païement	Stripe (Embedded Checkout)	Formulaire intégré, abonnements récurrents, notifications signées.
Exécution	Docker, FrankenPHP, Caddy	Environnements reproductibles, HTTPS automatique.
Tests	PHPUnit 12	Standard de fait pour PHP, intégration native avec Symfony.

Tableau 2 — Pile technique de CINAF v2

8.1. Symfony et API Platform

L'écosystème Symfony offre une combinaison rare de maturité, de composants découplés et de respect des standards du web (Symfony, 2025). API Platform, posé par-dessus, apporte la documentation de l'API au format OpenAPI, une pagination normalisée et la possibilité de sortir du cadre strict de ses opérations automatiques pour écrire des contrôleurs sur mesure (API Platform, 2025). Le projet a largement profité de cette souplesse : la plupart des points d'entrée métier sont des contrôleurs écrits à la main, que j'ai ensuite documentés un par un pour que la documentation OpenAPI reste complète, comme le montre le chapitre 18.

8.2. PostgreSQL plutôt que MySQL

Le cahier des charges prévoyait MySQL 8 ; le choix final s'est porté sur PostgreSQL 16, pour trois raisons techniques. La première est le type JSON natif, utilisé notamment pour la liste des rôles d'un utilisateur. La deuxième, décisive, est l'index unique partiel : il permet d'exprimer en une ligne qu'une seule demande de retrait peut être en attente pour une œuvre donnée, ce que MySQL ne sait pas faire directement. La troisième est la rigueur de PostgreSQL en matière de transactions et de contraintes (PostgreSQL Global Development Group, 2024). L'épisode illustre un principe sain : une décision d'analyse peut, et parfois doit, être révisée à la lumière de la réalisation.

8.3. Next.js et React

Côté interface, Next.js 14 a été retenu pour son routage par fichiers, son organisation en mises en page imbriquées et ses optimisations automatiques (Vercel, 2024). Il s'appuie sur React, bibliothèque de référence pour les interfaces à base de composants (Meta, 2024), et sur Bootstrap pour la grille et les composants de base, complété par un jeu de variables CSS qui porte l'identité visuelle de CINAF : fond sombre, accents dorés et typographie soignée.

8.4. Technologies envisagées puis écartées

Plusieurs alternatives ont été examinées. Pour l'authentification, la session classique a été écartée parce qu'elle lie l'utilisateur à un serveur et convient mal à une API destinée aussi à une application mobile. Le JWT signé par une clé symétrique a été écarté parce qu'une seule clé y sert à signer et à vérifier, de sorte qu'une fuite permet de fabriquer des jetons. La délégation complète par OAuth 2.0 aurait été une sur-ingénierie pour ce périmètre. Pour le stockage vidéo, AWS S3 et Firebase, cités dans le cahier des charges, ont cédé la place à Bunny.net : l'entreprise y héberge déjà son catalogue, et le service combine stockage et diffusion à un coût bien inférieur. Enfin, un serveur Node.js aurait obligé à réécrire des briques que Symfony fournit déjà.

9. Sécurité : une défense en profondeur

La sécurité a été pensée comme un empilement de protections indépendantes plutôt que comme une barrière unique. Une requête franchit successivement le filtrage des origines autorisées, la vérification du jeton, le contrôle d'accès global déclaré dans la configuration, l'attribut de contrôle posé sur chaque méthode de contrôleur, puis, pour les studios, la vérification de propriété assurée par un service dédié. Elle n'atteint qu'ensuite la logique métier et la base de données, interrogée exclusivement par des requêtes préparées. Un studio qui tenterait de modifier l'œuvre d'un autre en devinant son identifiant franchirait les premières couches, puisqu'il est authentifié et créateur, mais serait arrêté par la vérification de propriété.

Le tableau suivant met en regard les principaux risques de la liste de référence de l'OWASP (OWASP Foundation, 2021) et les protections en place, y compris celles qui restent à renforcer.

Risque (OWASP Top 10)	Protection mise en place
Contrôle d'accès défaillant	Hiérarchie de rôles, contrôle global et par méthode, vérification de propriété des œuvres et des chemins de fichiers.

Risque (OWASP Top 10)	Protection mise en place
Défaillances cryptographiques	Jetons signés en RS256, mots de passe hachés par l'algorithme recommandé par Symfony, HTTPS imposé par Caddy.
Injection	Doctrine et requêtes préparées pour tous les accès aux données.
Conception non sécurisée	Règles métier garanties par la base (unicités, index partiel), séparation des pouvoirs de l'administrateur.
Mauvaise configuration	Liste blanche des origines autorisées, fournie par variable d'environnement et sans joker.
Identification et authentification	Réponse identique que le compte existe ou non lors de l'oubli du mot de passe ; rotation du jeton de rafraîchissement. À renforcer : aucune limitation du nombre de tentatives de connexion.
Intégrité des données	Notifications de paiement authentifiées par signature HMAC horodatée ; types et extensions de fichiers contrôlés au téléversement.

Tableau 3 — Risques OWASP et protections

Pour les paiements, le serveur recalcule la signature de chaque notification Stripe à partir du corps brut de la requête et de son horodatage, et rejette toute notification fausse ou trop ancienne (Stripe, 2025) : une requête forgée ne peut pas activer un abonnement.

10. Intégrations tierces

10.1. Bunny.net

Bunny.net assure le stockage et la diffusion des fichiers (Bunny.net, 2025), organisés en zones. CINAF en utilise plusieurs : la zone qui contient le catalogue vidéo de l'entreprise, où chaque épisode est un dossier contenant déjà ses rendus HLS, et la zone qui héberge les affiches. Le serveur accède aux zones par l'API de stockage, pour téléverser les fichiers des studios en flux continu, jusqu'à plusieurs gigaoctets, ou pour parcourir le stockage depuis l'administration. Un registre de zones centralise les identifiants et les adresses de chaque zone, de sorte qu'aucun contrôleur ne manipule directement une clé d'accès.

10.2. Stripe

Stripe gère tout ce qui touche à l'argent. Les deux formules d'abonnement sont décrites dans l'application et synchronisées avec Stripe par une commande dédiée. La carte est saisie dans un cadre fourni par Stripe et affiché dans la page d'abonnement ; les événements de la vie d'un abonnement (paiement, renouvellement, résiliation) parviennent ensuite au serveur par des notifications signées. Un mode simulé, activé par une variable d'environnement, permet enfin de faire tourner l'application sans compte Stripe, pour une démonstration hors ligne.

PARTIE III — RÉALISATION TECHNIQUE

Cette partie raconte la construction de CINAF v2. Elle suit l'ordre réel du développement, sprint après sprint, puis la phase de consolidation qui a suivi. Elle s'attarde volontiers sur ce qui a résisté : c'est souvent là, plus que dans le code qui fonctionne du premier coup, que se joue la pratique professionnelle.

11. Méthodologie de développement

Le projet s'est étalé de mars à septembre 2026. Le cahier des charges et le dossier d'analyse ont été rédigés en mars. Le développement proprement dit s'est déroulé d'avril à mai, en sprints d'environ une semaine, chacun centré sur une fonctionnalité cohérente et clôturé par des tests automatisés. Les mois d'été ont ensuite été consacrés à la consolidation : importer le vrai catalogue, corriger ce que l'usage révélait, documenter et préparer le déploiement.

Sprint	Thème	Principaux résultats
1	Authentification et utilisateurs	Jetons JWT signés, rafraîchissement à rotation, profil, export et suppression des données.
2	Catalogue et diffusion	Entités du catalogue, double source de données, pages publiques, premier lecteur.
3	Abonnement et paiement	Formules, paiement Stripe intégré, notifications signées, résiliation différée.
4	Studios de production	Espace studio, téléversement vers Bunny, modération du premier contenu, retraits.
5	Lecteur et diffusion	Contrôle du droit de lecture, lecteur HLS et ses solutions de repli.
6	Notation et commentaires	Reporté : non réalisé dans cette version.
7	Administration	Comptes et rôles, abonnements, approbations, retraits, explorateur du stockage.
Été	Consolidation	Import du catalogue réel et des affiches, refonte de l'accueil, documentation complète de l'API, tests, déploiement.

Tableau 4 — Déroulement du développement

Trois habitudes ont accompagné tout le projet. Le code est versionné avec Git. Chaque fonctionnalité livrée s'accompagne de tests qui protègent contre les régressions. Enfin, j'ai tenu à jour des fichiers de mémoire interne et une série de guides techniques, qui consignent les décisions, les pièges rencontrés et la manière de les éviter. Travailler seul sur plusieurs mois, c'est aussi devoir se réexpliquer régulièrement son propre code : ces notes m'ont fait gagner un temps considérable.

12. Sprint 1 — Authentification et utilisateurs

Le premier sprint a posé les fondations de la sécurité. Sans authentification fiable, rien d'autre ne tient : l'abonnement, les studios et la modération reposent tous sur la capacité du serveur à savoir, avec certitude, qui envoie une requête.

Les jetons d'accès sont signés en RS256, c'est-à-dire avec une paire de clés asymétriques : la clé privée, qui ne quitte pas le serveur, signe les jetons, et la clé publique permet de les

vérifier. Un jeton d'accès vit quinze minutes. Il est complété par un jeton de rafraîchissement valable trente jours, à usage unique : chaque renouvellement en délivre un nouveau et invalide l'ancien, de sorte qu'un jeton volé puis réutilisé devient détectable.

Une attention particulière a été portée à une faille classique, la référence directe non sécurisée à un objet : un utilisateur qui remplace l'identifiant présent dans l'adresse par celui d'un autre ne doit rien obtenir. Chaque action sur un compte vérifie donc que la cible est bien l'utilisateur connecté, comme dans cet extrait de l'export des données personnelles :

```
if ($user->getUserIdentifier() !== $this->getUser()->getUserIdentifier()) {  
    return $this->json(['message' => 'Accès refusé.'], 403);  
}
```

Extrait de UserController::exportData()

Le module couvre l'inscription, la connexion, le rafraîchissement et la déconnexion, la réinitialisation du mot de passe, la gestion du profil, ainsi que l'export et la suppression des données au titre du RGPD. L'inscription ne connecte pas l'utilisateur : elle crée le compte et génère un lien de vérification. La demande de réinitialisation renvoie toujours la même réponse, que l'adresse existe ou non, pour ne pas permettre de deviner quels comptes existent.

Ce sprint a aussi laissé des dettes, consignées dès le départ : aucune limitation du nombre de tentatives de connexion, une règle de mot de passe limitée à huit caractères minimum, et un envoi de courriels qui, en développement, passe par un transport fictif. Le lien de vérification est bien produit côté serveur, mais la page de l'interface qui doit l'accueillir reste à écrire.

13. Sprint 2 — Catalogue et diffusion vidéo

Le deuxième sprint a introduit le catalogue : les entités présentées dans l'analyse, les points d'entrée publics et les premières pages de l'interface. C'est aussi le sprint qui a le plus évolué par la suite, parce qu'il a fallu le confronter au vrai catalogue de l'entreprise.

13.1. Un catalogue, deux sources

Le catalogue public peut être servi à partir de deux sources, choisies par une variable de configuration. La première lit directement l'arborescence du stockage Bunny et en déduit les œuvres ; la seconde interroge la base de données. Ce double mode a été précieux pendant la migration : il permettait de comparer, œuvre par œuvre, ce que voyait chaque source. Aujourd'hui, la base de données est la source de référence ; la lecture directe du stockage reste disponible comme outil de diagnostic.

13.2. Importer le catalogue réel

Le stockage de l'entreprise ne contient aucune métadonnée : seulement des dossiers et des fichiers vidéo. Il fallait donc déduire, à partir de la seule structure des dossiers, ce qui est un film et ce qui est une série. Ma première approche était une heuristique : un dossier qui contient des sous-dossiers ressemblant à des saisons est une série, les autres sont des films. Sur quatre-vingt-seize œuvres, elle produisait un résultat plausible, mais elle s'est révélée fragile : un long métrage découpé en plusieurs fichiers ressemblait à s'y méprendre à une série en plusieurs épisodes.

La solution retenue s'appuie sur l'emplacement plutôt que sur la forme. Dans le stockage, les films sont rangés sous un dossier FILMS ; tout ce qui se trouve à la racine est une série. Chaque sous-dossier de FILMS devient un film, et ses vidéos deviennent ses parties. Les bandes-annonces sont reconnues et rattachées à l'œuvre au lieu d'être prises pour des épisodes, et les ré-encodages en double, repérés par un suffixe _F, sont ignorés lorsque l'original existe. La commande d'import est idempotente : relancée, elle ignore les œuvres déjà présentes et ne touche jamais aux contenus créés par les studios. Chaque œuvre importée est rattachée de façon déterministe à l'un des dix studios de démonstration.

Le résultat donne une idée du volume traité : 185 œuvres, soit 98 films et 87 séries, qui totalisent 117 parties de films, 147 saisons et 3 780 épisodes.

13.3. Les affiches : relier ce qui n'a pas de clé

Les vidéos étaient importées, mais le catalogue restait gris : aucune affiche. Les visuels existent pourtant, dans une autre zone de stockage de l'entreprise. Le problème est qu'ils y sont rangés sous des noms génériques, construits à partir des identifiants internes du site public cinaf.tv, du type `titles/{titleId}/poster/{assetId}.png`. Rien, dans ces noms, ne permet de savoir à quel film ils appartiennent, et je ne disposais pas de la table de correspondance qui relie ces identifiants aux œuvres. Les vidéos, elles, sont identifiées par le nom de leur dossier. Les deux systèmes ne partagent aucune clé commune.

Le premier obstacle a d'ailleurs été plus terre à terre. L'API de stockage répondait systématiquement « 401 non autorisé » sur cette zone, ce qui orientait vers une clé d'accès erronée. La clé était bonne : la zone est hébergée à Stockholm et doit être interrogée par le point d'accès régional correspondant. Le message d'erreur, trompeur, m'a coûté un bon moment de vérifications inutiles.

Pour relier les affiches aux œuvres, j'ai utilisé le seul pont disponible : le plan du site public de cinaf.tv, qui associe chacun des 214 identifiants de titre à un nom lisible. Une commande

dédiée rapproche ces noms des titres en base, en trois passes de confiance décroissante : correspondance exacte, inclusion, puis similarité au-delà d'un seuil. Elle a attribué une affiche à 138 œuvres sur 185 : 74 par correspondance exacte, 39 par inclusion et 25 par similarité. Les 64 correspondances non exactes ont été listées dans un rapport à relire, et les 47 œuvres restées sans visuel ont reçu une affiche de remplacement aux couleurs de CINAF.

NOTE SUR LES VISUELS DE LA DÉMONSTRATION

Dans la version de démonstration, certaines affiches ne correspondent pas au film ou à la série qu'elles illustrent. Ce n'est pas une erreur de l'application : c'est la conséquence directe du stockage des images sous des noms génériques, sans la clé de correspondance qui aurait permis de les rattacher à coup sûr à leur œuvre.

J'ai corrigé à la main une partie des rapprochements douteux. Pour les autres, j'ai préféré compléter le catalogue avec des visuels attribués arbitrairement, afin que la démonstration reste agréable à parcourir plutôt que d'afficher des dizaines de cartes vides. En production, ce problème disparaît de lui-même : il suffit d'obtenir la table de correspondance de cinaf.tv, et les studios téléversent de toute façon leurs propres affiches, que l'application associe directement à leurs œuvres.

13.4. Côté interface

Ce sprint a livré les pages publiques : l'accueil, le catalogue, les listes et les fiches des films et des séries, la recherche par titre et l'annuaire des studios. L'accueil a ensuite été repensé pendant la consolidation, sous forme de carrousels thématiques inspirés du site de l'entreprise. Un composant d'affiche gère le cas où l'image manque ou ne se charge pas, en affichant un visuel de repli plutôt qu'une image cassée.

14. Sprint 3 — Abonnement et paiement

Le troisième sprint a doté la plateforme de son modèle économique : un abonnement proposé au mois, à 9,99 €, ou à l'année, à 99 €. Le formulaire de carte s'affiche directement dans la page d'abonnement, ce qui évite de perdre l'utilisateur sur un site externe au moment le plus sensible du parcours.

Une souscription se déroule ainsi. L'interface demande au serveur d'ouvrir une session de paiement ; le serveur la crée auprès de Stripe, en y attachant l'identifiant de l'utilisateur et de la formule choisie, et renvoie un secret que l'interface utilise pour afficher le formulaire. Une fois le paiement accepté, Stripe ramène l'utilisateur sur une page de confirmation et envoie, de son côté, une notification signée au serveur.

C'est ici qu'un défaut sérieux a été découvert en fin de projet. À l'origine, seule la notification de Stripe activait l'abonnement. En production, c'est suffisant. En développement, en revanche, Stripe ne peut pas joindre un serveur local sans un outil de relais lancé à part : sans ce relais, le paiement était bien encaissé, mais l'abonnement ne s'activait jamais, et l'utilisateur restait bloqué devant son écran de confirmation. Le défaut était invisible dans les tests automatisés, qui simulent la notification, et ne se manifestait qu'en conditions réelles.

La correction a consisté à activer aussi l'abonnement au retour du client. La page de confirmation interroge le serveur, qui récupère la session directement auprès de Stripe, vérifie qu'elle appartient bien à l'utilisateur connecté et qu'elle est payée, puis applique exactement le même traitement que la notification :

```
$metadataUserId = $session->metadata->toArray()['user_id'] ?? null;
if ($metadataUserId === null
    || (string) $metadataUserId !== $user->getId()->toRfc4122()) {
    return false;
}
if ($session->status !== 'complete'
    || !\in_array($session->payment_status,
                ['paid', 'no_payment_required'], true)) {
    return false;
}

return $this->handleCheckoutCompleted($session);
```

Extrait de SubscriptionService::fulfillCheckoutSession() (mise en forme adaptée à la page)

Les deux chemins peuvent désormais arriver dans n'importe quel ordre, voire en même temps. Le traitement commun est idempotent, et un index unique sur l'identifiant d'abonnement Stripe garantit, au niveau de la base, qu'un même paiement ne crée jamais deux abonnements.

La résiliation, enfin, est différée : résilier ne coupe pas l'accès, mais programme la fin de l'abonnement au terme de la période payée. L'utilisateur garde l'accès jusque-là et peut revenir sur sa décision. Ce comportement, fidèle à celui des services d'abonnement réels, a demandé de distinguer soigneusement l'état stocké en base et l'état perçu par l'utilisateur.

15. Sprint 4 — Module des studios de production

Le quatrième sprint est le plus ambitieux. Il transforme CINAF d'une plateforme de consommation en un espace à deux versants, où les studios fournissent les œuvres et les abonnés les regardent. Une vingtaine de points d'entrée composent l'espace studio, tous protégés par les couches de sécurité décrites au chapitre 9.

N'importe quel membre peut devenir producteur en libre-service. Pour éviter qu'un nouveau venu ne publie immédiatement un contenu inapproprié, le premier contenu d'un studio

passer par la modération, selon le processus décrit au chapitre 5 : l'administrateur l'approuve, ce qui valide le studio, et les contenus suivants sont publiés directement. Ce garde-fou, ajouté en cours de projet, concilie ouverture et contrôle.

Les fichiers suivent une convention de chemins stricte, « un dossier par projet » : l'affiche, la bande-annonce et la vidéo d'une œuvre sont rangées sous le dossier du studio, avec des noms déterministes. Le serveur refuse toute référence à un fichier situé hors du dossier du studio, ce qui ferme la porte à l'appropriation du contenu d'autrui. Une phase de durcissement a complété le sprint : contrôle du type MIME et de l'extension de chaque fichier téléversé, pour qu'un exécutable déguisé en image soit refusé même si le navigateur le présente comme tel, et vérification systématique de la propriété des œuvres.

Côté base de données, la règle « une seule demande de retrait en attente par œuvre » est confiée à PostgreSQL. Doctrine ne sait pas générer d'index partiel à partir des attributs des entités (Doctrine Project, 2025) ; la migration l'ajoute donc en SQL :

```
CREATE UNIQUE INDEX uniq_withdrawal_pending
ON withdrawal_request (target_type, target_id)
WHERE status = 'PENDING'
```

Index unique partiel créé par la migration Version20260430100200

16. Sprint 5 — Lecteur vidéo et diffusion

Le cinquième sprint s'est concentré sur la lecture. Avant d'afficher le lecteur, l'interface demande au serveur si l'utilisateur a le droit de lire l'œuvre. La réponse, un simple oui ou non, repose sur l'abonnement actif en base. Un visiteur non connecté est invité à se connecter ; un membre sans abonnement voit une invitation à souscrire.

Les vidéos sont diffusées au format HLS, qui découpe un film en courts segments disponibles en plusieurs qualités et laisse le lecteur choisir celle que permet la connexion (PANTOS & MAY, 2017). Le lecteur prend en charge trois situations : la lecture HLS native, sous Safari ; la lecture HLS par la bibliothèque hls.js (video-dev, 2025), sur les autres navigateurs ; et un repli sur un fichier MP4 lorsque le flux adaptatif est indisponible et qu'un tel fichier existe. Les bandes-annonces, accessibles à tous, s'ouvrent dans une fenêtre superposée, depuis les fiches comme depuis les carrousels de l'accueil.

Deux limites importantes ont été identifiées et documentées plutôt que masquées. La première concerne la protection des flux : le droit de lecture est vérifié, mais les adresses des vidéos ne sont pas signées. Quelqu'un qui connaît l'adresse d'un flux peut donc le lire sans

abonnement. Pour une mise en production, il faudra activer la signature des adresses par un jeton de courte durée, généré par le serveur pour chaque lecture.

La seconde est plus surprenante. Une vidéo téléversée par un studio est bien stockée et accessible sur le réseau de diffusion, mais le lecteur ne parvient pas à la lire. La cause tient en une ligne : l'adresse de lecture est construite en ajoutant `/master.m3u8` au chemin enregistré. Pour le catalogue importé, ce chemin désigne un dossier qui contient déjà un manifeste HLS, et tout fonctionne. Pour un studio, il désigne un fichier MP4 : l'adresse obtenue ne correspond à rien. Le module studio et le catalogue importé ont été construits à des moments différents, avec des conventions différentes, et rien ne les a encore réconciliés. Deux corrections sont possibles : transcoder les vidéos des studios en HLS, comme le catalogue importé, ou, plus simplement dans un premier temps, transmettre l'adresse du fichier MP4 au lecteur, qui sait déjà s'en servir.

17. Sprint 7 — Administration de la plateforme

Le sprint 6, prévu pour la notation et les commentaires, a été reporté afin de consolider l'existant ; il figure parmi les perspectives. Le sprint 7 a livré l'interface d'administration, en plusieurs phases. Elle regroupe la gestion des comptes (recherche, suspension, modification des rôles, suppression, export au format CSV), la gestion des abonnements des membres (résiliation et réactivation), la supervision des films et des séries, la file d'approbation des premiers contenus, le traitement des demandes de retrait et un explorateur du stockage Bunny.

Une protection empêche un administrateur de se supprimer, de se suspendre ou de se retirer à lui-même ses droits d'administration. Sans elle, une fausse manipulation pourrait priver la plateforme de tout administrateur :

```
if ($this->isSelf($target)) {  
    return $this->json(  
        ['message' => 'Vous ne pouvez pas supprimer votre propre compte.'],  
        400  
    );  
}
```

Extrait de `AdminController::delete()` (mise en forme adaptée à la page)

18. Organisation du serveur et documentation de l'API

À la fin du projet, le serveur compte vingt-quatre contrôleurs, répartis entre les trois contextes : onze pour le contexte public et utilisateur, cinq pour les studios et huit pour l'administration. Ils s'appuient sur huit services, dont trois propres au contexte studio, quatre

commandes en ligne de commande (import du catalogue et des affiches, exploration du stockage, synchronisation des formules Stripe) et quinze migrations de base de données.

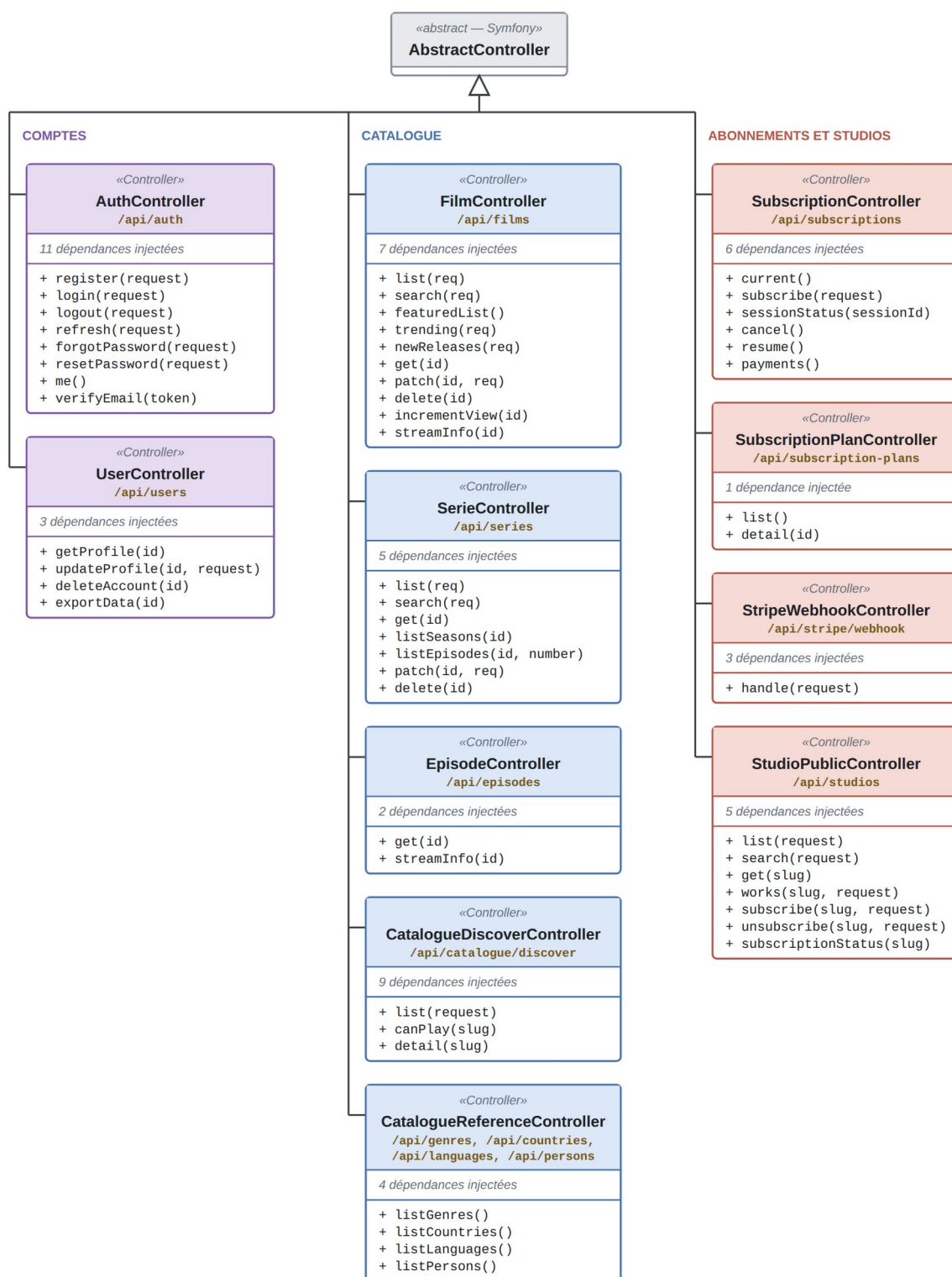


Figure 10 — Les onze contrôleurs du contexte public et utilisateur

La figure 10 présente les contrôleurs du contexte public, regroupés par domaine. Tous héritent du contrôleur de base de Symfony, déclarent leur préfixe de route et reçoivent leurs

dépendances par injection ; chacune de leurs méthodes publiques correspond à un point d'entrée de l'API et renvoie une réponse JSON. Les contrôleurs des studios et de l'administration suivent exactement la même structure.

Puisque la plupart des points d'entrée sont des contrôleurs écrits à la main, la documentation générée automatiquement par API Platform ne les décrivait pas. Pendant la consolidation, chacun a été documenté : la documentation OpenAPI couvre aujourd'hui 118 opérations sur 89 chemins, réparties en 22 groupes, dont 88 exigent un jeton et 30 sont publiques. Un test automatisé compare les routes déclarées dans le code et celles de la documentation : si un développeur ajoute un point d'entrée sans le documenter, le test échoue. Dans le même esprit, l'ensemble du code serveur a été commenté, fichier par fichier.

Contexte	Préfixe	Accès	Exemples
Public et utilisateur	/api/auth, /api/catalogue, /api/studios, /api/subscriptions	public ou jeton	connexion, catalogue, pages des studios, abonnement
Studio	/api/studio	jeton, ROLE_CREATEUR	œuvres, saisons, téléversement, retraits
Administration	/api/admin	jeton, ROLE_ADMIN	comptes, approbations, retraits, stockage
Paieement	/api/stripe/webhook	signature Stripe	notifications de paiement

Tableau 5 — Organisation des points d'entrée de l'API

19. L'interface utilisateur

L'interface compte trente-six pages, organisées en trois espaces étanches. La Vitrine, publique, réunit le catalogue, la lecture, le compte et l'abonnement. L'espace studio et l'administration sont protégés par une mise en page commune qui vérifie le rôle de l'utilisateur et le renvoie vers la connexion s'il n'a rien à y faire. Cette vérification côté interface relève du confort : la véritable barrière reste le serveur, qui contrôle chaque requête.

Dans l'arborescence de la figure 11, chaque page porte un identifiant (V pour la Vitrine, S pour le studio, A pour l'administration), utilisé dans la documentation interne. Deux vues techniques complètent l'ensemble sans figurer dans l'arborescence : la page d'erreur, pour une adresse inconnue, et une vue dédiée aux œuvres retirées. Pour une œuvre retirée, l'API ne répond pas « introuvable » (404) mais « n'existe plus » (410) ; l'interface reconnaît ce code et explique au visiteur que le contenu a été retiré de la plateforme, plutôt que de lui laisser croire à une adresse erronée.

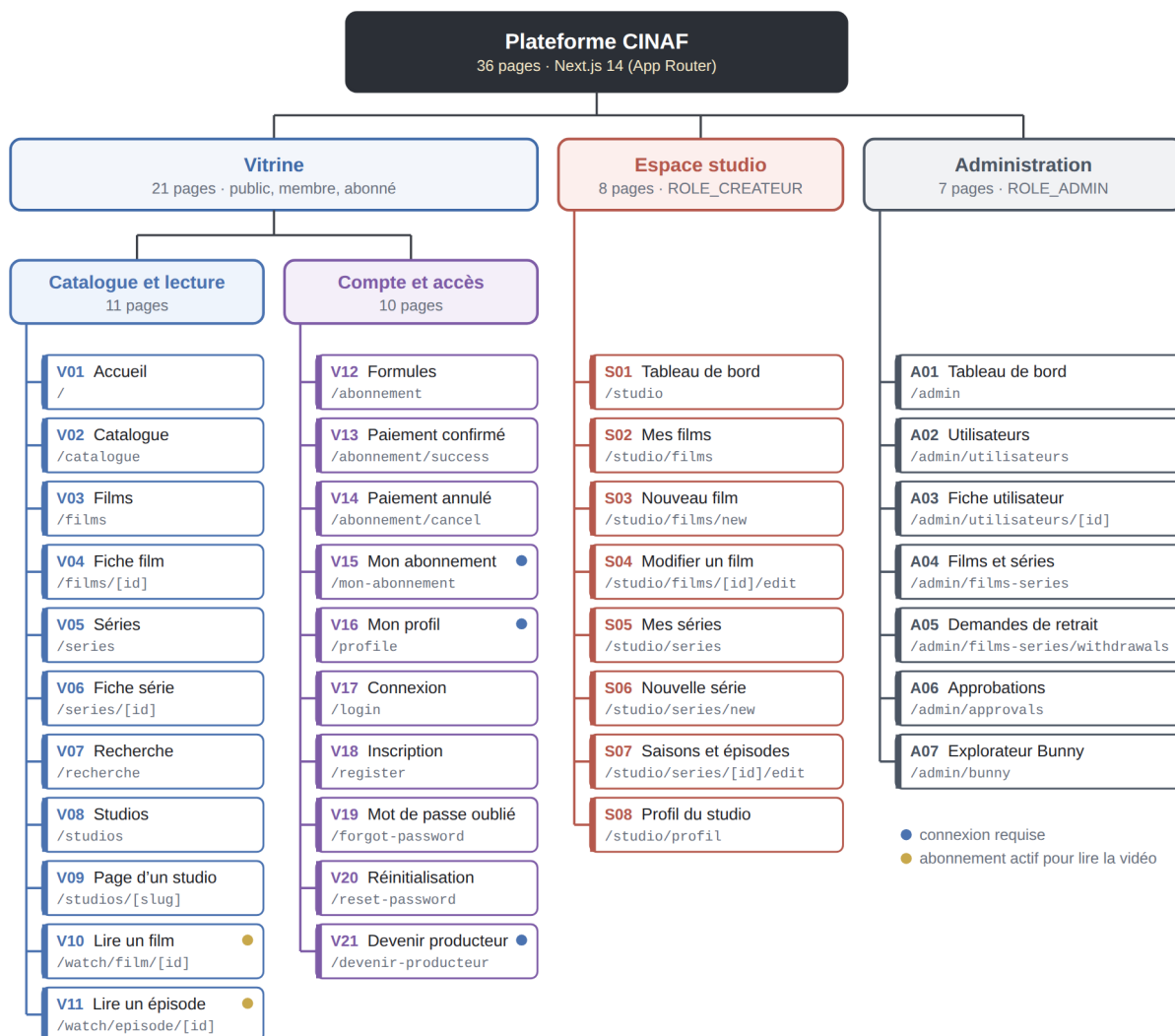


Figure 11 — Arborescence des trente-six pages de l'interface

20. Tests automatisés

La qualité du serveur est soutenue par une suite de 255 tests, qui totalisent 1 046 assertions et passaient sans échec lors de la dernière exécution, le 26 septembre 2026. Elle combine des tests unitaires des services, des tests fonctionnels des contrôleurs, qui envoient de vraies requêtes HTTP à l'application, et des tests de bout en bout qui rejouent des scénarios complets.

Le scénario le plus représentatif suit la vie entière d'une œuvre : un studio la crée, la publie, en demande le retrait, puis l'administrateur accepte ce retrait et l'œuvre disparaît du catalogue public. Ce seul test traverse cinq contrôleurs et trois services. L'épisode du paiement, raconté au chapitre 14, rappelle cependant les limites de l'exercice : un test qui simule un service externe ne vérifie que ce que l'on a pensé à simuler.

21. Difficultés rencontrées

21.1. Un environnement de développement exigeant

L'environnement Windows a été une source de friction régulière. PHP n'y trouve pas de magasin de certificats système : tous les appels HTTPS vers Bunny.net, Stripe ou le site de l'entreprise échouaient tant qu'un fichier de certificats racine n'était pas fourni explicitement, d'où une option dédiée dans plusieurs services et commandes. S'y sont ajoutés des problèmes de droits sur les dossiers de cache et d'encodage des caractères accentués dans les scripts.

Un défaut précis a coûté du temps à plusieurs reprises. À chaque génération de migration, Doctrine proposait de supprimer la table des jetons de rafraîchissement, qu'il ne reconnaissait pas comme la sienne parce qu'elle appartient à un module externe. Oublier de retirer cette ligne rendait toute connexion impossible après la migration. Le problème s'est reproduit trois fois avant d'être noté en tête des fichiers de mémoire, puis dans les migrations elles-mêmes ; il n'est plus revenu depuis.

21.2. Des performances difficiles à mesurer

En développement, chaque requête mettait environ une seconde à aboutir, même pour une réponse triviale. Plutôt que d'optimiser à l'aveugle, j'ai cherché l'origine de ce coût fixe, qui tient à l'environnement plus qu'au code : le projet se trouvait dans un dossier synchronisé par OneDrive, l'antivirus analysait chaque fichier lu par PHP, le débogueur Xdebug était actif et le serveur de développement ne disposait que d'un seul processus. Dans ces conditions, l'objectif de 500 ms du cahier des charges ne pouvait pas être mesuré honnêtement.

Certaines optimisations valaient néanmoins d'être faites, car elles profiteront aussi à la production : une mise en cache de soixante secondes des réponses publiques du catalogue, le calcul en une seule requête des compteurs affichés dans l'annuaire des studios, au lieu d'une requête par studio, et l'activation du cache d'opcodes de PHP dans l'image de production.

21.3. Un chantier abandonné : le transcodage par Bunny Stream

Pour rendre lisibles les vidéos des studios, j'ai étudié leur envoi automatique à Bunny Stream, le service de transcodage de Bunny.net. Le travail a été spécifié en détail dans un guide interne, puis le développement a été arrêté et le code retiré : le chantier touchait à la fois au téléversement, au stockage et au lecteur, et le mener à terme si près de la fin mettait en danger la stabilité de la version livrée. Ce retour en arrière n'est pas un échec. C'est une décision d'ingénierie assumée, qui laisse une spécification prête à l'emploi pour la suite.

Conclusion

Ce travail avait pour objectif de concevoir et de réaliser CINAF v2, une plateforme de streaming consacrée au cinéma africain indépendant, capable de démontrer une chaîne de valeur complète : un visiteur qui s'inscrit, s'abonne et regarde un film ; un membre qui devient producteur, publie son premier contenu et le voit approuvé par un administrateur. Cet objectif est atteint, et il l'est sur un catalogue réel de 185 œuvres plutôt que sur quelques données d'exemple.

La première partie a posé l'analyse : le besoin, les quatre acteurs et leurs droits emboîtés, les cas d'utilisation, un modèle de dix-sept entités traduit en vingt-six tables, et trois processus métiers redessinés pour décrire fidèlement l'application. La deuxième a justifié l'architecture découplée, le monolithe organisé en trois contextes, la pile technique et ses écarts assumés avec le cahier des charges (PostgreSQL plutôt que MySQL, Bunny.net plutôt qu'AWS), ainsi qu'une sécurité pensée en couches successives. La troisième a raconté la réalisation, des jetons signés du premier sprint jusqu'à la documentation complète de l'API, en passant par les difficultés qui m'ont le plus appris : l'import d'un catalogue sans métadonnées, des affiches sans clé de correspondance, ou un paiement encaissé sans que l'abonnement s'active.

Ce travail ne prétend pas livrer un produit fini, et ses limites sont connues. Les vidéos téléversées par les studios ne sont pas encore lisibles, faute d'un raccordement entre le stockage et le lecteur. Les adresses des flux vidéo ne sont pas signées. Le nombre de tentatives de connexion n'est pas limité. L'envoi réel des courriels et la page de confirmation d'adresse restent à finaliser, tout comme la gestion des paiements échoués. La reprise de lecture, la notation et les commentaires n'ont pas été réalisés, et certaines affiches de la démonstration ne correspondent pas à leur œuvre. Les connaître et les documenter fait partie de la démarche : un logiciel professionnel se juge aussi à la lucidité avec laquelle son auteur en voit les angles morts.

Les perspectives découlent en partie de ces limites. À court terme, il s'agit de rendre lisibles les vidéos des studios, d'abord en transmettant le fichier MP4 au lecteur, puis en reprenant la spécification du transcodage par Bunny Stream ; de signer les adresses des flux ; de limiter les tentatives de connexion et de brancher un véritable service d'envoi de courriels. À moyen terme viendront la reprise de lecture, la notation et les commentaires, des statistiques d'audience pour les studios et une mesure sérieuse des performances sur l'infrastructure de production. À plus long terme, l'architecture découplée rend possible une application mobile qui réutiliserait la même API, et la rémunération des studios selon l'audience donnerait tout son sens à la dimension de place de marché du projet.

Sur le plan personnel, ce projet a mobilisé bien plus que la programmation : l'analyse, l'architecture, la sécurité, la gestion d'un projet mené seul sur plusieurs mois et la rédaction d'une documentation utile. Le plus difficile n'a pas été une brique technique en particulier, chacune restant abordable isolément : c'était de faire travailler ensemble des technologies et des services nombreux, dans un environnement contraint, en acceptant de revenir en arrière quand c'était la décision raisonnable. C'est l'enseignement le plus précieux que je retire de ce travail.

Bibliographie

Les références ci-dessous, présentées selon les normes APA, recensent les sources effectivement citées dans ce travail.

API Platform. (2025). *API Platform documentation (version 4.1)*.

<https://api-platform.com/docs/> (consulté en septembre 2026).

Bunny.net. (2025). *Bunny Storage and CDN documentation*. <https://docs.bunny.net/> (consulté en septembre 2026).

Caddy. (2025). *Automatic HTTPS*. Caddy 2 documentation.

<https://caddyserver.com/docs/automatic-https> (consulté en septembre 2026).

Doctrine Project. (2025). *Doctrine ORM documentation (version 3.x)*. <https://www.doctrine-project.org/projects/orm.html> (consulté en septembre 2026).

JONES, M., BRADLEY, J., & SAKIMURA, N. (2015). *JSON Web Token (JWT)* (RFC 7519).

Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc7519>

Meta. (2024). *React documentation*. <https://react.dev/> (consulté en septembre 2026).

Object Management Group. (2011). *Business Process Model and Notation (BPMN), version 2.0*. <https://www.omg.org/spec/BPMN/2.0/>

Object Management Group. (2017). *Unified Modeling Language (UML), version 2.5.1*.

<https://www.omg.org/spec/UML/2.5.1/>

OWASP Foundation. (2021). *OWASP Top 10:2021*. <https://owasp.org/Top10/> (consulté en septembre 2026).

PANTOS, R., & MAY, W. (2017). *HTTP Live Streaming* (RFC 8216). Internet Engineering

Task Force. <https://www.rfc-editor.org/rfc/rfc8216>

PostgreSQL Global Development Group. (2024). *PostgreSQL 16 documentation*.

<https://www.postgresql.org/docs/16/> (consulté en septembre 2026).

Stripe. (2025). *Stripe API and Embedded Checkout documentation*. <https://docs.stripe.com/> (consulté en septembre 2026).

Symfony. (2025). *Symfony 7 documentation*. <https://symfony.com/doc/current/> (consulté en septembre 2026).

Union européenne. (2016). Règlement (UE) 2016/679 du Parlement européen et du Conseil du 27 avril 2016 relatif à la protection des personnes physiques à l'égard du traitement

des données à caractère personnel et à la libre circulation de ces données. *Journal officiel de l'Union européenne*, L 119, 1-88.

Vercel. (2024). *Next.js documentation (App Router)*. <https://nextjs.org/docs> (consulté en septembre 2026).

video-dev. (2025). *hls.js : HLS client in JavaScript*. GitHub. <https://github.com/video-dev/hls.js> (consulté en septembre 2026).

Annexes

Les annexes sont paginées en chiffres romains. Le tableau suivant en donne la liste.

Annexe A — Le cahier des charges et sa mise en œuvre.....	ii
Annexe B — Le dossier d'analyse.....	iv
Annexe C — Dictionnaire des données.....	v
Annexe D — Livrables électroniques.....	viii
Annexe E — Lexique.....	ix

Annexe A — Le cahier des charges et sa mise en œuvre

Le cahier des charges de CINAF v2 (version 1.0 du 27 mars 2026) est fourni en version électronique³. Plutôt que d'en reproduire le contenu, cette annexe le confronte à l'application livrée. Chaque exigence reçoit un statut : réalisée telle quelle, adaptée en cours de route, réalisée en partie, reportée ou non réalisée.

Exigence du cahier des charges	Statut	Dans l'application
AUTHENTIFICATION		
Inscription avec lien de confirmation	Partiel	Lien généré par le serveur ; envoi réel des courriels et page de confirmation à finaliser.
Connexion JWT et jeton de rafraîchissement	Réalisé	Jeton de 15 min, rafraîchissement de 30 jours à rotation (7 jours prévus).
Réinitialisation du mot de passe	Réalisé	Réponse identique que l'adresse existe ou non.
RGPD : consentement, portabilité, droit à l'oubli	Réalisé	Consentement à l'inscription, export des données, suppression du compte.
Rôles visiteur, abonné, créateur, modérateur, administrateur	Adapté	L'état d'abonné découle de l'abonnement actif ; le rôle de modérateur existe sans écran dédié.
CATALOGUE		
Catalogue paginé, fiches, séries en saisons et épisodes	Réalisé	185 œuvres importées du stockage de l'entreprise.
Recherche plein texte et filtres avancés	Partiel	Recherche par titre, filtre film ou série.
Contenu éditorial (tendances, nouveautés)	Réalisé	Carrousels de l'accueil et mise en avant d'œuvres.
ABONNEMENT ET PAIEMENT		
Formules mensuelle et annuelle, paiement Stripe	Réalisé	9,99 € par mois ou 99 € par an, formulaire intégré.
Souscription, renouvellement, annulation	Réalisé	Renouvellement par Stripe, résiliation différée et réactivation.
Historique des transactions	Réalisé	Historique des paiements fourni par Stripe.
Période d'essai configurable	Non réalisé	Aucune période d'essai.
Accès au contenu selon l'abonnement	Réalisé	Droit de lecture lié à l'abonnement actif en base.
STUDIOS DE PRODUCTION		
Profil du studio	Réalisé	Nom, description, logo ; page publique avec compteur d'abonnés.
Téléversement vidéo vers AWS S3 ou Firebase	Adapté	MP4, MOV et WebM vers Bunny Storage ; lecture à raccorder (chapitre 16).
Gestion du catalogue du studio	Réalisé	Brouillons, publication, saisons et épisodes, retrait sur demande motivée.

³Fichier *CAHIER_DES_CHARGES_CINAF_V2.docx*, dossier *DOC* du dépôt.

Exigence du cahier des charges	Statut	Dans l'application
Restrictions géographiques, droits et licences	Non réalisé	—
Tableau de bord analytique	Partiel	Compteurs d'œuvres, de brouillons, d'abonnés et de retraits ; pas de statistiques d'audience.
ADMINISTRATION		
Gestion des utilisateurs	Réalisé	Recherche, suspension, rôles, suppression, export CSV.
Validation, mise en avant et retrait de contenus	Réalisé	File d'approbation et demandes de retrait.
Validation des studios	Réalisé	Par l'approbation du premier contenu.
Indicateurs et rapports CSV ou PDF	Partiel	Export CSV des comptes ; écran de statistiques annoncé.
Modération des commentaires	Reporté	Liée à la notation et aux commentaires.
LECTEUR VIDÉO		
Contrôles standard et streaming adaptatif	Réalisé	HLS natif ou via hls.js, repli MP4.
Choix de la qualité	Adapté	Qualité adaptée automatiquement au débit.
Reprise de lecture, sous-titres	Non réalisé	—
NOTATION ET COMMENTAIRES		
Notes, avis, signalements	Reporté	Sprint 6 reporté.
CHOIX TECHNIQUES ET LIVRABLES		
MySQL 8	Adapté	PostgreSQL 16 (chapitre 8).
Nginx devant PHP	Adapté	Caddy et FrankenPHP.
Conteneurisation Docker, HTTPS	Réalisé	Docker Compose, certificats Let's Encrypt automatiques.
Documentation de l'API	Réalisé	118 opérations OpenAPI, test de couverture.
Application déployée en ligne	Partiel	Configuration de déploiement prête et documentée.

Tableau 6 — Correspondance entre le cahier des charges et l'application livrée

Annexe B — Le dossier d'analyse

Le dossier d'analyse de CINAF v2 (version 1.0, mars 2026) est fourni en version électronique⁴. Il présente le contexte, les acteurs, les diagrammes de cas d'utilisation et de classes par module, le modèle de données et l'ensemble des exigences. Il décrit surtout vingt-huit processus métiers détaillés, répartis en sept modules :

- Authentification : PM-01 à PM-05 ;
- Abonnement et paiement : PM-06 à PM-09 ;
- Catalogue : PM-10 à PM-13 ;
- Lecteur vidéo : PM-14 à PM-16 ;
- Studio de production : PM-17 à PM-21 ;
- Modération : PM-22 à PM-24 ;
- Administration : PM-25 à PM-28.

Ce dossier a été rédigé avant le développement. Il garde donc la trace d'intentions qui ont évolué, comme le choix de MySQL ou un rôle d'abonné distinct. Les diagrammes de la première partie du présent rapport, redessinés à partir du code, décrivent l'application telle qu'elle est livrée ; en cas de divergence, ce sont eux qui font foi, et l'annexe A récapitule les écarts.

⁴Fichier *DOSSIER_ANALYSE_CINAF_V2.pdf*, dossier *DOC/Rapport TFE* du dépôt.

Annexe C — Dictionnaire des données

Ce dictionnaire décrit les vingt-six tables de la base PostgreSQL, regroupées par domaine comme dans le schéma de la figure 4. La table technique des versions de migration, gérée par Doctrine, n'y figure pas. La mention « nul » signale une colonne facultative. Les identifiants sont des UUID, sauf pour la table des jetons de rafraîchissement, qui appartient à un module externe.

Table	Colonnes et types	Clés et contraintes
CATALOGUE — FILMS		
film	id uuid · title varchar(255) · slug varchar(280) · synopsis text · year int · duration int · poster varchar(500) (nul) · trailer_video_id varchar(500) (nul) · bunny_video_id varchar(500) (nul) · bunny_folder varchar(500) (nul) · views int · avg_rating float · studio_id uuid (nul) · status varchar(20) · published_at timestamp (nul) · withdrawn_at timestamp (nul) · created_at timestamp · updated_at timestamp	PK : id FK : studio_id → studio Unique : slug
film_part	id uuid · film_id uuid · number int · title varchar(255) · bunny_video_id varchar(500)	PK : id FK : film_id → film Unique : (film_id, number)
film_director	film_id uuid · person_id uuid	PK : film_id, person_id FK : film_id → film ; person_id → person
film_cast	film_id uuid · person_id uuid	PK : film_id, person_id FK : film_id → film ; person_id → person
film_genre	film_id uuid · genre_id uuid	PK : film_id, genre_id FK : film_id → film ; genre_id → genre
film_country	film_id uuid · country_id uuid	PK : film_id, country_id FK : film_id → film ; country_id → country
film_tag	film_id uuid · tag_id uuid	PK : film_id, tag_id FK : film_id → film ; tag_id → tag
CATALOGUE — SÉRIES		
serie	id uuid · title varchar(255) · slug varchar(280) · synopsis text · year int · poster varchar(500) (nul) · trailer_video_id varchar(500) (nul) · bunny_folder varchar(500) (nul) · nb_seasons int · studio_id uuid (nul) · status varchar(20) · published_at timestamp (nul) · withdrawn_at timestamp (nul) · created_at timestamp · updated_at timestamp	PK : id FK : studio_id → studio Unique : slug
season	id uuid · serie_id uuid · number int · title varchar(255) (nul) · synopsis text (nul)	PK : id FK : serie_id → serie Unique : (serie_id, number)
episode	id uuid · season_id uuid · number int · title varchar(255) · synopsis text (nul) · duration int · bunny_video_id varchar(500) (nul)	PK : id FK : season_id → season Unique : (season_id, number)

Table	Colonnes et types	Clés et contraintes
serie_genre	serie_id uuid · genre_id uuid	PK : serie_id, genre_id FK : serie_id → serie ; genre_id → genre
serie_country	serie_id uuid · country_id uuid	PK : serie_id, country_id FK : serie_id → serie ; country_id → country
serie_tag	serie_id uuid · tag_id uuid	PK : serie_id, tag_id FK : serie_id → serie ; tag_id → tag
RÉFÉRENTIELS PARTAGÉS		
person	id uuid · first_name varchar(100) · last_name varchar(100) · photo varchar(500) (nul) · biography text (nul) · birth_date date (nul) · created_at timestamp · updated_at timestamp	PK : id
genre	id uuid · name varchar(100) · slug varchar(120) · created_at timestamp · updated_at timestamp	PK : id Unique : name ; slug
country	id uuid · name varchar(100) · iso_code varchar(3) · created_at timestamp · updated_at timestamp	PK : id Unique : iso_code
tag	id uuid · name varchar(100) · slug varchar(120)	PK : id Unique : name ; slug
STUDIOS ET MODÉRATION		
studio	id uuid · name varchar(255) · slug varchar(280) · owner_id uuid · description text (nul) · logo_url varchar(500) (nul) · bunny_folder varchar(255) · is_active bool · is_validated bool · created_at timestamp · updated_at timestamp	PK : id FK : owner_id → user Unique : name ; slug ; owner_id ; bunny_folder
studio_subscription	id uuid · user_id uuid · studio_id uuid · subscribed_at timestamp	PK : id FK : user_id → user ; studio_id → studio Unique : (user_id, studio_id)
withdrawal_request	id uuid · studio_id uuid · requested_by_id uuid · target_type varchar(10) · target_id uuid · reason text · status varchar(20) · reviewed_by_id uuid (nul) · reviewed_at timestamp (nul) · review_note text (nul) · created_at timestamp	PK : id FK : studio_id → studio ; requested_by_id → user ; reviewed_by_id → user Unique : (target_type, target_id) si status = PENDING
COMPTES ET ABONNEMENTS		
user	id uuid · email varchar(180) · roles json · password varchar · first_name varchar(100) · last_name varchar(100) · is_verified bool · is_suspended bool · verification_token varchar(255) (nul) · password_reset_token varchar(255) (nul) · password_reset_token_expiry timestamp (nul) · consent_rgpd bool (nul) · created_at timestamp · updated_at timestamp	PK : id Unique : email
subscription	id uuid · user_id uuid · plan_id uuid · status varchar(20) · starts_at timestamp · ends_at timestamp (nul) · canceled_at timestamp (nul) · stripe_subscription_id varchar(100) (nul) · stripe_customer_id varchar(100) (nul) · created_at timestamp · updated_at timestamp	PK : id FK : user_id → user ; plan_id → subscription_plan Unique : stripe_subscription_id
subscription_plan	id uuid · name varchar(100) · description text (nul) · price_cents int · currency varchar(3) · interval_unit varchar(10) · interval_count int · features json · is_active bool · stripe_price_id varchar(100) (nul) · created_at timestamp · updated_at timestamp	PK : id

Table	Colonnes et types	Clés et contraintes
DIVERS ET INFRASTRUCTURE		
featured_content	id uuid · film_id uuid (nul) · serie_id uuid (nul) · position int · start_date timestamp · end_date timestamp (nul) · active bool	PK : id FK : film_id → film ; serie_id → serie
refresh_tokens	id int (serial) · refresh_token varchar · username varchar · valid timestamp	PK : id Unique : refresh_token
language	id uuid · name varchar(100) · iso_code varchar(5) · created_at timestamp · updated_at timestamp	PK : id Unique : iso_code

Tableau 7 — Dictionnaire des données

Annexe D — Livrables électroniques

Les éléments suivants accompagnent ce rapport sous forme électronique. Les diagrammes imprimés dans le rapport sont des versions redessinées pour l'impression ; les fichiers sources ci-dessous en sont les versions de travail, interactives pour la plupart.

Élément	Fichier ou dossier	Contenu
Code source	backend/, frontend/	Serveur Symfony et interface Next.js, avec leurs tests.
Déploiement	compose.prod.yaml, Caddyfile, DEPLOY.md	Conteneurs, reverse proxy et procédure de déploiement.
Cahier des charges	DOC/CAHIER_DES_CHARGES_CINAF_V2.docx	Version 1.0 du 27 mars 2026 (annexe A).
Dossier d'analyse	DOC/Rapport TFE/DOSSIER_ANALYSE_CINAF_V2.pdf	Version 1.0 de mars 2026 (annexe B).
Cas d'utilisation	DOC/diagramme-cas-utilisation-cinaf.html	Diagramme UML interactif.
Diagrammes de classes	DOC/diagramme-de-classes-cinaf.html, DOC/diagramme-classes-controller-cinaf.html	Entités et contrôleurs.
Paquetages	DOC/diagramme-paquetages-cinaf.html	Organisation du code.
Arborescence	DOC/arborescence-plateforme-cinaf.html	Les trente-six pages de l'interface.
Processus métiers	DOC/processus-plateforme-cinaf.bpmn (et .html, .svg)	Modèle BPMN 2.0 de synthèse, ouvrable dans Camunda Modeler.
Schéma relationnel	DOC/schema-relationnel-bdd.drawio, DOC/schema-relationnel-bdd.html	Schéma complet des vingt-six tables.
Guides techniques	DOC/MEMORY/guide/	Sept guides : authentification, pipeline Bunny, intégration Stripe, création de contenu...
Affiches à relire	DOC/AUDIT/affiches-a-relire.txt	Rapprochements d'affiches non exacts (chapitre 13).

Tableau 8 — Livrables électroniques

Annexe E — Lexique

Ce lexique rassemble, par ordre alphabétique, les termes techniques employés dans le rapport.

Terme	Définition
API REST	Interface par laquelle l'interface et le serveur échangent des données au format JSON, à travers des adresses HTTP.
BPMN	Notation normalisée de modélisation des processus métiers.
CDN	Réseau de serveurs répartis dans le monde qui délivre les fichiers au plus près de l'utilisateur.
CORS	Mécanisme par lequel un serveur indique quelles origines (sites) ont le droit de l'interroger depuis un navigateur.
Docker	Outil d'emballage d'une application et de ses dépendances dans des conteneurs reproductibles.
HLS	Format de diffusion vidéo qui découpe un film en segments disponibles en plusieurs qualités.
Idempotent	Se dit d'un traitement qui produit le même résultat qu'il soit exécuté une ou plusieurs fois.
Index unique partiel	Contrainte d'unicité qui ne s'applique qu'aux lignes vérifiant une condition.
JWT	Jeton signé qui transporte l'identité de l'utilisateur à chaque requête.
Migration	Script versionné qui fait évoluer le schéma de la base de données.
OpenAPI	Format standard de description d'une API, lisible par les humains et par les outils.
ORM	Couche qui fait correspondre les objets du code et les tables de la base de données.
RGPD	Règlement européen sur la protection des données personnelles.
Sprint	Période courte de développement consacrée à un ensemble cohérent de fonctionnalités.
Transcodage	Conversion d'une vidéo dans d'autres formats ou qualités.
UML	Langage de modélisation graphique des systèmes logiciels.
UUID	Identifiant unique de 128 bits, généré sans consulter la base de données.
Webhook	Notification envoyée automatiquement par un service externe à une adresse du serveur.